

University of Mumbai

PRACTICAL JOURNAL



612

Natural Language Processing

SUBMITTED BY

Muhammad Tarique Ansari

SEAT NO: _____

SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS

FOR QUALIFYING M.Sc.IT (Artificial Intelligence)PART-II
(SEMESTER – IV) EXAMINATION

Department of Information Technology
2nd floor, IT park University of Mumbai,
Vidyanagari, Santacruz (East),
Mumbai 400098

2025-26

University of Mumbai



Department of Information

Technology

Certificate

This is to certify that Mr./Ms. **Muhammad Tarique Ansari**, Seat No. Studying in **M.Sc.IT (Artificial Intelligence) Part II Semester IV** has satisfactorily completed the Practical of **612 Natural Language Processing** as prescribed by University of Mumbai, during the academic year **2025 - 26**.

Signature

Subject-In-Charge

Signature

Head of the Department

Signature External Examiner

College Seal: _____

Date: _____

Index

Sr.no	Practical Description	Date	Page no	sign
1.	a. Install NLTK b. Convert the given text to speech c. Convert audio file Speech to Text.	28/01/2026	1-6	
2.	a. Study of various Corpus – Brown, Inaugural, Reuters, udhr with various methods like fields, raw, words, sents, categories. b. Create and use your own corpora(plaintext, categorical) c. Study Conditional frequency distributions Study of tagged corpora with methods like tagged_sents, tagged_words. d. Write a program to find the most frequent noun tags. e. Map Words to Properties Using Python Dictionaries f. Study DefaultTagger, Regular expression tagger, UnigramTagger. g. Find different words from a given plain text without any space by comparing this text with a given corpus of words. Also find the score of words.	04/02/2026	7-22	
3.	a. Study of Wordnet Dictionary with methods as synsets, definitions, examples, antonyms. b. Study lemmas, hyponyms, hypernyms, entailments. c. Write a program using python to find synonym and antonym of word "active" using Wordnet. d. Compare two nouns e. Handling stopword. f. Using nltk Adding or Removing Stop Words in NLTK's Default Stop Word List Using Gensim Adding and Removing Stop Words in Default Gensim Stop Words List. g. Using Spacy Adding and Removing Stop Words in Default Spacy Stop Words List	11/02/2026	23-32	
4.	Text Tokenization a. Tokenization using Python's split() function b. Tokenization using Regular Expressions (RegEx) c. Tokenization using NLTK d. Tokenization using the spaCy library e. Tokenization using Keras f. Tokenization using Gensim	18/02/2026	33-38	

5.	Important NLP Libraries for Indian Languages and perform: - word tokenization in Hindi - Generate similar sentences from a given Hindi text input - Identify the Indian language of a text	25/02/2026	39-42	
6.	Illustrate part of speech tagging. - Part of speech Tagging and chunking of user defined text. - Named Entity recognition of user defined text. - Named Entity recognition with diagram using NLTK corpus – treebank	11/03/2026	43-46	
7.	- Define grammar using nltk. Analyze a sentence using the same. - Accept the input string with Regular expression of FA: 101+ - Accept the input string with Regular expression of FA: (a+b)*bba - Implementation of Deductive Chart Parsing using context free grammar and a given sentence.	18/03/2026	47-53	
8.	- Study PorterStemmer, LancasterStemmer, RegexpStemmer, SnowballStemmer - Study WordNetLemmatizer	25/03/2026	54-56	
9.	- Implement Naive Bayes classifier - Speech Tagging: - Speech tagging using spacy - Speech tagging using - nltk Statistical parsing: - Usage of Give and Gave in the Penn Treebank sample - Probabilistic	01/04/2026	57-59	
10.	- parser Malt parsing: - Parse a sentence and draw a tree using malt parsing. - Multiword Expressions in NLP - Normalized Web Distance and Word Similarity - Word Sense Disambiguation	01/04/2026	60-65	
11.	- Multiword Expression in NLP - Normalized Web Distance and Word Similarity - Word Sense Disambiguation	08/04/2026	66-70	

Practical 1

Aim(A): Study different libraries used for NLP in python.

Theory:

1. NLTK (Natural Language Toolkit):

- Use: NLTK is a powerful platform for building Python programs to work with human language data.
- Features:
 - i. Tokenization: Splits text into words or sentences.
 - ii. Stemming and Lemmatization: Reduces words to their base forms.
 - iii. Part-of-speech tagging: Identifies grammatical components.
 - iv. Parsing: Analyzes sentence structure.
 - v. Commonly Used in: NLP research and education.

2. spaCy:

- Use: spaCy is a fast and efficient library for NLP tasks.
- Features
 - i. Tokenization, part-of-speech tagging, and named entity recognition.
 - ii. Dependency parsing.
 - iii. Pre-trained word vectors.
 - iv. Commonly Used in: Production environments due to its speed.

3. TextBlob:

- Use: TextBlob is beginner-friendly and built on NLTK.
- Features:
 - i. Sentiment analysis.
 - ii. Part-of-speech tagging.
 - iii. Noun phrase extraction.
 - iv. Commonly Used in: Simple NLP tasks.

4. Gensim:

- Use: Gensim focuses on topic modeling and document similarity.
- Features:
 - i. Word embeddings (Word2Vec).
 - ii. Latent Semantic Analysis (LSA).
 - iii. Latent Dirichlet Allocation (LDA).
 - iv. Commonly Used in: Topic modeling and semantic analysis.

5. VADER (Valence Aware Dictionary and Sentiment Reasoner) :

- Use: VADER is specifically designed for sentiment analysis.
- Features:

- i. Pre-trained lexicon for sentiment scoring.
- ii. Handles sentiment intensity and context.
- iii. Commonly Used in: Sentiment analysis of social media data.

6. AllenNLP :

- a. Use: AllenNLP is for deep learning-based NLP.
- b. Features:

Customizable deep learning models.

- i. Easy experimentation.
- ii. Commonly Used in: Research and custom model development.

7. Stanza:

- a. Use: Stanza provides pre-trained models for various languages.
- b. Features:
 - i. Tokenization, part-of-speech tagging, and dependency parsing.
 - ii. Named entity recognition.
 - iii. Commonly Used in: Multilingual NLP.

8. FastText:

- a. Use: FastText is efficient for word embeddings.
- b. Features:
 - i. Word vectors.
 - ii. Text classification.
 - iii. Commonly Used in: Word representation and text classification.

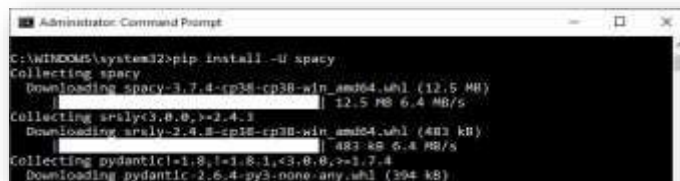
Installation of different NLP Packages:

1. Gensim



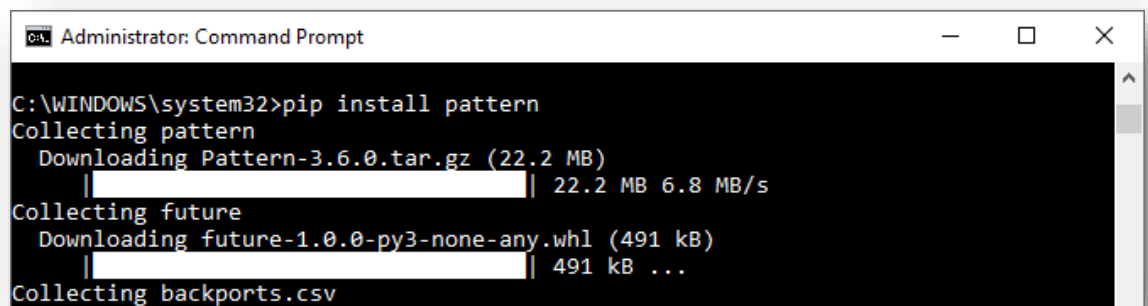
```
Administrator: Command Prompt - pip install gensim
C:\WINDOWS\system32>pip install gensim
Collecting gensim
  Downloading gensim-4.3.2-cp38-cp38-win_amd64.whl (24.0 MB)
    | 24.0 MB 89 kB/s
Collecting scipy>=1.7.0
  Downloading scipy-1.10.1-cp38-cp38-win_amd64.whl (42.2 MB)
    | 42.2 MB 6.8 MB/s
Collecting smart-open>=1.8.1
  Downloading smart-open-7.0.4-py3-none-any.whl (61 kB)
    | 61 kB 1.9 MB/s
Collecting numpy>=1.18.5
  Downloading numpy-1.24.4-cp38-cp38-win_amd64.whl (14.9 MB)
    | 14.9 MB 6.8 MB/s
Collecting wrapt
  Downloading wrapt-1.16.0-cp38-cp38-win_amd64.whl (37 kB)
Installing collected packages: numpy, scipy, wrapt, smart-open, gensim
```

2. Spacy



```
Administrator: Command Prompt
C:\WINDOWS\system32>pip install -U spacy
Collecting spacy
  Downloading spacy-3.7.4-cp38-cp38-win_amd64.whl (12.5 MB)
    | 12.5 MB 6.4 MB/s
Collecting srsly<3.0.0,=>2.4.3
  Downloading srsly-2.4.3-cp38-cp38-win_amd64.whl (483 kB)
    | 483 kB 6.4 MB/s
Collecting pydantic<1.8.1,=>1.8.1,<3.0.0,=>1.7.4
  Downloading pydantic-2.6.4-py3-none-any.whl (394 kB)
```

3. Pattern



```
Administrator: Command Prompt
C:\WINDOWS\system32>pip install pattern
Collecting pattern
  Downloading Pattern-3.6.0.tar.gz (22.2 MB)
    | 22.2 MB 6.8 MB/s
Collecting future
  Downloading future-1.0.0-py3-none-any.whl (491 kB)
    | 491 kB ...
Collecting backports.csv
```

Practical 1

Aim(B): Install NLTK Packages.

Theory:

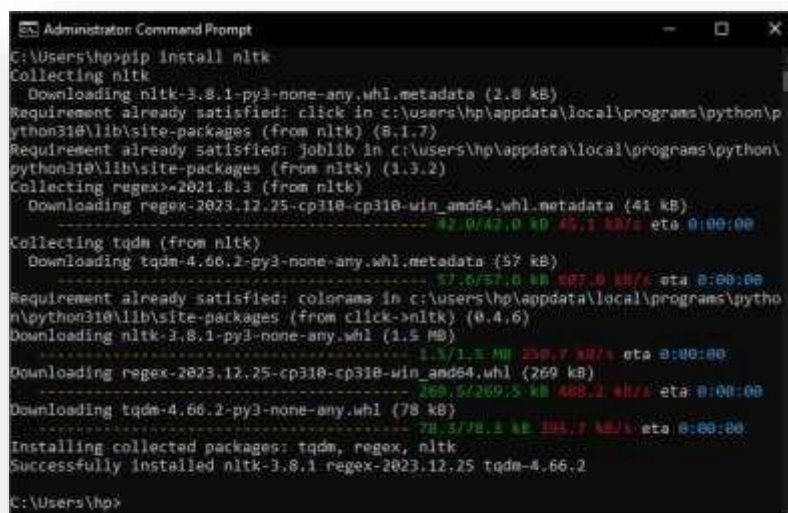
NLTK (Natural Language Toolkit) is a popular Python library used for Natural Language Processing (NLP). It provides many tools and packages that help computers understand and process human language such as English text or speech.

NLTK is widely used for text analysis, language modeling, machine learning, and research in NLP.

Source Code:

```
pip install nltk
```

Output:



```
Administrator: Command Prompt
C:\Users\hp>pip install nltk
Collecting nltk
  Downloading nltk-3.8.1-py3-none-any.whl.metadata (2.8 kB)
Requirement already satisfied: click in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from nltk) (8.1.7)
Requirement already satisfied: joblib in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from nltk) (1.3.2)
Collecting regex>=2021.8.3 (from nltk)
  Downloading regex-2023.12.25-cp310-cp310-win_amd64.whl.metadata (41 kB)
----- 42.0/42.0 kB 45.1 kB/s eta 0:00:00
Collecting tqdm (from nltk)
  Downloading tqdm-4.66.2-py3-none-any.whl.metadata (57 kB)
----- 37.6/37.6 kB 407.0 kB/s eta 0:00:00
Requirement already satisfied: colorama in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from click->nltk) (0.4.6)
Downloading nltk-3.8.1-py3-none-any.whl (1.5 MB)
----- 1.5/1.5 MB 250.7 kB/s eta 0:00:00
Downloading regex-2023.12.25-cp310-cp310-win_amd64.whl (269 kB)
----- 269.5/269.5 kB 408.2 kB/s eta 0:00:00
Downloading tqdm-4.66.2-py3-none-any.whl (78 kB)
----- 78.5/78.5 kB 205.7 kB/s eta 0:00:00
Installing collected packages: tqdm, regex, nltk
Successfully installed nltk-3.8.1 regex-2023.12.25 tqdm-4.66.2
C:\Users\hp>
```

Conclusion:

In this practical, we successfully installed the NLTK packages and required datasets. This setup enables us to perform various NLP tasks like tokenization, tagging, and text processing efficiently.

Practical 1

Aim(C): Convert the given text into speech.

Theory:

In this practical, we use Python's gTTS (Google Text-to-Speech) library to convert a given text into audible speech. The text is first stored in a variable, and the language is set to English. Using the gTTS class, the text is processed and transformed into a speech object, which is then saved as an MP3 file on the local system.

The slow parameter allows controlling the speed of speech, where False produces normal-speed audio. Finally, using IPython.display.Audio, the generated audio can be played directly within a Jupyter Notebook or Google Colab environment.

This practical demonstrates how text can be vocalized using programming, which is useful in applications such as screen readers, virtual assistants, and automated announcements, bridging the gap between written and spoken communication.

Source Code:

```
from gtts import gTTS

from IPython.display import Audio

import os

mytext = "Welcome to Practical 1: Natural Language Programming, Muhammad  
Tarique"
language = "en"
filename = "Tarique.mp3"

myobj = gTTS(text=mytext, lang=language, slow=False)

myobj.save(filename)

Audio(filename)
```

Output:



Conclusion:

In this practical, we successfully converted text into speech using Python libraries. This demonstrates how written text can be transformed into audible output, which is useful in applications like voice assistants and accessibility tools.

Practical 1

Aim(D): Convert the audio file into speech.

Theory:

In this practical, we convert an audio file into text using Python libraries Speech Recognition and pydub.

The audio file, such as a .wav recording, is first loaded into the program using AudioFile from the Speech Recognition library.

The Recognizer class is then used to process the audio, capturing the spoken words in the file. Using Google's speech recognition API through the recognize_google method, the audio data is transcribed into text, which can be printed or saved for further use.

This process is useful in applications such as voice assistants, automated transcription services, and accessibility tools for converting spoken language into written text. The practical demonstrates how computers can interpret and convert human speech into readable text, bridging the gap between audio and textual data.

Source Code:

```
pip install SpeechRecognition pydub

import speech_recognition as sr

from pydub import AudioSegment

filename = "Recording.wav"

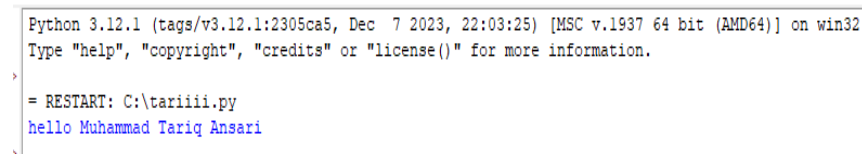
r = sr.Recognizer()

with sr.AudioFile(filename) as source:
    audio_data = r.record(source)

    text = r.recognize_google(audio_data)

print(text)
```

Output:



```
Python 3.12.1 (tags/v3.12.1:2305ca5, Dec 7 2023, 22:03:25) [MSC v.1937 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>
= RESTART: C:\tariiii.py
hello Muhammad Tariq Ansari
>
```

Conclusion:

In this practical, we successfully converted an audio file into text using speech recognition. This demonstrates how spoken language can be transformed into readable text, which is

useful in applications like voice assistants, transcription, and accessibility tools.

Practical 2

Aim(A): Study of various corpus – Brown, Inaugural, Reuters, udhr with various methods like fields, raw, words, sents, categories.

Theory:

The study of various corpora such as the Brown, Inaugural, Reuters, and UDHR corpora is an important practical in Natural Language Processing using the NLTK library. A corpus is a structured collection of text used for analyzing language patterns.

The Brown Corpus is one of the earliest and most balanced corpora, containing texts from different categories like news, fiction, and romance, and it is useful for general language analysis.

The Inaugural Corpus consists of U.S. presidential speeches, which helps in studying political language and historical trends over time. The Reuters Corpus contains a large collection of news articles categorized into topics such as trade, money, and crude, making it highly useful for text classification tasks.

Methods to Explore Corpora (using NLTK)

- raw()
 - Returns the **complete text as a single string**
- words()
 - Returns a **list of individual words**
- sents()
 - Returns the text divided into **sentences**
- categories()
 - Shows **different categories or genres**
 - Available in corpora like **Brown and Reuters**
- fileids()
 - Returns **document names or file IDs**
- fields()
 - Used in **structured corpora**
 - Helps access **specific sections of data**

Source code:

```
import nltk

nltk.download('brown')

from nltk.corpus import brown

# Display file IDs of Brown corpus
print('File ids of brown corpus:\n', brown.fileids())

# Pick a sample file
ca01 = brown.words('ca01')

# Display first few words
print('\nca01 has following words:\n', ca01)

# Total number of words
print('\nca01 has', len(ca01), 'words')

# Categories or files
print('\n\nCategories or files in brown corpus:\n')
print(brown.categories())

# Display statistics for each text
print('\n\nStatistics for each text:\n')
print('AvgWordLen\tAvgSentenceLen\tNo.ofTimesEachWordAppearsOnAvg\tFileName')

for fileid in brown.fileids():
    num_chars = len(brown.raw(fileid))
    num_words = len(brown.words(fileid))
    num_sents = len(brown.sents(fileid))
    num_vocab = len(set([w.lower() for w in brown.words(fileid)]))
    print(
        int(num_chars/num_words), "\t\t",
        int(num_words/num_sents), "\t\t",
        int(num_words/num_vocab), "\t\t",
        fileid
    )
```

Output:

```
[nltk_data] Downloading package brown to
[nltk_data]   C:\Users\Alfija\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\brown.zip.
File ids of brown corpus:
['ca01', 'ca02', 'ca03', 'ca04', 'ca05', 'ca06', 'ca07', 'ca08', 'ca09', 'ca10', 'ca11', 'ca12', 'ca13', 'ca14', 'ca15', 'ca16', 'ca17', 'ca18', 'ca19', 'ca20', 'ca21', 'ca22', 'ca23', 'ca24', 'ca25', 'ca26', 'ca27', 'ca28', 'ca29', 'ca30', 'ca31', 'ca32', 'ca33', 'ca34', 'ca35', 'ca36', 'ca37', 'ca38', 'ca39', 'ca40', 'ca41', 'ca42', 'ca43', 'ca44', 'ca45', 'ca46', 'ca47', 'ca48', 'ca49', 'ca50', 'ca51', 'ca52', 'ca53', 'ca54', 'ca55', 'ca56', 'ca57', 'ca58', 'ca59', 'ca60', 'ca61', 'ca62', 'ca63', 'ca64', 'ca65', 'ca66', 'ca67', 'ca68', 'ca69', 'ca70', 'ca71', 'ca72', 'ca73', 'ca74', 'ca75', 'ca76', 'ca77', 'ca78', 'ca79', 'ca80', 'ca81', 'ca82', 'ca83', 'ca84', 'ca85', 'ca86', 'ca87', 'ca88', 'ca89', 'ca90', 'ca91', 'ca92', 'ca93', 'ca94', 'ca95', 'ca96', 'ca97', 'ca98', 'ca99', 'ca100', 'ca101', 'ca102', 'ca103', 'ca104', 'ca105', 'ca106', 'ca107', 'ca108', 'ca109', 'ca110', 'ca111', 'ca112', 'ca113', 'ca114', 'ca115', 'ca116', 'ca117', 'ca118', 'ca119', 'ca120', 'ca121', 'ca122', 'ca123', 'ca124', 'ca125', 'ca126', 'ca127', 'ca128', 'ca129', 'ca130', 'ca131', 'ca132', 'ca133', 'ca134', 'ca135', 'ca136', 'ca137', 'ca138', 'ca139', 'ca140', 'ca141', 'ca142', 'ca143', 'ca144', 'ca145', 'ca146', 'ca147', 'ca148', 'ca149', 'ca150', 'ca151', 'ca152', 'ca153', 'ca154', 'ca155', 'ca156', 'ca157', 'ca158', 'ca159', 'ca160', 'ca161', 'ca162', 'ca163', 'ca164', 'ca165', 'ca166', 'ca167', 'ca168', 'ca169', 'ca170', 'ca171', 'ca172', 'ca173', 'ca174', 'ca175', 'ca176', 'ca177', 'ca178', 'ca179', 'ca180', 'ca181', 'ca182', 'ca183', 'ca184', 'ca185', 'ca186', 'ca187', 'ca188', 'ca189', 'ca190', 'ca191', 'ca192', 'ca193', 'ca194', 'ca195', 'ca196', 'ca197', 'ca198', 'ca199', 'ca200', 'ca201', 'ca202', 'ca203', 'ca204', 'ca205', 'ca206', 'ca207', 'ca208', 'ca209', 'ca210', 'ca211', 'ca212', 'ca213', 'ca214', 'ca215', 'ca216', 'ca217', 'ca218', 'ca219', 'ca220', 'ca221', 'ca222', 'ca223', 'ca224', 'ca225', 'ca226', 'ca227', 'ca228', 'ca229', 'ca230', 'ca231', 'ca232', 'ca233', 'ca234', 'ca235', 'ca236', 'ca237', 'ca238', 'ca239', 'ca240', 'ca241', 'ca242', 'ca243', 'ca244', 'ca245', 'ca246', 'ca247', 'ca248', 'ca249', 'ca250', 'ca251', 'ca252', 'ca253', 'ca254', 'ca255', 'ca256', 'ca257', 'ca258', 'ca259', 'ca260', 'ca261', 'ca262', 'ca263', 'ca264', 'ca265', 'ca266', 'ca267', 'ca268', 'ca269', 'ca270', 'ca271', 'ca272', 'ca273', 'ca274', 'ca275', 'ca276', 'ca277', 'ca278', 'ca279', 'ca280', 'ca281', 'ca282', 'ca283', 'ca284', 'ca285', 'ca286', 'ca287', 'ca288', 'ca289', 'ca290', 'ca291', 'ca292', 'ca293', 'ca294', 'ca295', 'ca296', 'ca297', 'ca298', 'ca299', 'ca300', 'ca301', 'ca302', 'ca303', 'ca304', 'ca305', 'ca306', 'ca307', 'ca308', 'ca309', 'ca310', 'ca311', 'ca312', 'ca313', 'ca314', 'ca315', 'ca316', 'ca317', 'ca318', 'ca319', 'ca320', 'ca321', 'ca322', 'ca323', 'ca324', 'ca325', 'ca326', 'ca327', 'ca328', 'ca329', 'ca330', 'ca331', 'ca332', 'ca333', 'ca334', 'ca335', 'ca336', 'ca337', 'ca338', 'ca339', 'ca340', 'ca341', 'ca342', 'ca343', 'ca344', 'ca345', 'ca346', 'ca347', 'ca348', 'ca349', 'ca350', 'ca351', 'ca352', 'ca353', 'ca354', 'ca355', 'ca356', 'ca357', 'ca358', 'ca359', 'ca360', 'ca361', 'ca362', 'ca363', 'ca364', 'ca365', 'ca366', 'ca367', 'ca368', 'ca369', 'ca370', 'ca371', 'ca372', 'ca373', 'ca374', 'ca375', 'ca376', 'ca377', 'ca378', 'ca379', 'ca380', 'ca381', 'ca382', 'ca383', 'ca384', 'ca385', 'ca386', 'ca387', 'ca388', 'ca389', 'ca390', 'ca391', 'ca392', 'ca393', 'ca394', 'ca395', 'ca396', 'ca397', 'ca398', 'ca399', 'ca400', 'ca401', 'ca402', 'ca403', 'ca404', 'ca405', 'ca406', 'ca407', 'ca408', 'ca409', 'ca410', 'ca411', 'ca412', 'ca413', 'ca414', 'ca415', 'ca416', 'ca417', 'ca418', 'ca419', 'ca420', 'ca421', 'ca422', 'ca423', 'ca424', 'ca425', 'ca426', 'ca427', 'ca428', 'ca429', 'ca430', 'ca431', 'ca432', 'ca433', 'ca434', 'ca435', 'ca436', 'ca437', 'ca438', 'ca439', 'ca440', 'ca441', 'ca442', 'ca443', 'ca444', 'ca445', 'ca446', 'ca447', 'ca448', 'ca449', 'ca450', 'ca451', 'ca452', 'ca453', 'ca454', 'ca455', 'ca456', 'ca457', 'ca458', 'ca459', 'ca460', 'ca461', 'ca462', 'ca463', 'ca464', 'ca465', 'ca466', 'ca467', 'ca468', 'ca469', 'ca470', 'ca471', 'ca472', 'ca473', 'ca474', 'ca475', 'ca476', 'ca477', 'ca478', 'ca479', 'ca480', 'ca481', 'ca482', 'ca483', 'ca484', 'ca485', 'ca486', 'ca487', 'ca488', 'ca489', 'ca490', 'ca491', 'ca492', 'ca493', 'ca494', 'ca495', 'ca496', 'ca497', 'ca498', 'ca499', 'ca500', 'ca501', 'ca502', 'ca503', 'ca504', 'ca505', 'ca506', 'ca507', 'ca508', 'ca509', 'ca510', 'ca511', 'ca512', 'ca513', 'ca514', 'ca515', 'ca516', 'ca517', 'ca518', 'ca519', 'ca520', 'ca521', 'ca522', 'ca523', 'ca524', 'ca525', 'ca526', 'ca527', 'ca528', 'ca529', 'ca530', 'ca531', 'ca532', 'ca533', 'ca534', 'ca535', 'ca536', 'ca537', 'ca538', 'ca539', 'ca540', 'ca541', 'ca542', 'ca543', 'ca544', 'ca545', 'ca546', 'ca547', 'ca548', 'ca549', 'ca550', 'ca551', 'ca552', 'ca553', 'ca554', 'ca555', 'ca556', 'ca557', 'ca558', 'ca559', 'ca560', 'ca561', 'ca562', 'ca563', 'ca564', 'ca565', 'ca566', 'ca567', 'ca568', 'ca569', 'ca570', 'ca571', 'ca572', 'ca573', 'ca574', 'ca575', 'ca576', 'ca577', 'ca578', 'ca579', 'ca580', 'ca581', 'ca582', 'ca583', 'ca584', 'ca585', 'ca586', 'ca587', 'ca588', 'ca589', 'ca590', 'ca591', 'ca592', 'ca593', 'ca594', 'ca595', 'ca596', 'ca597', 'ca598', 'ca599', 'ca600', 'ca601', 'ca602', 'ca603', 'ca604', 'ca605', 'ca606', 'ca607', 'ca608', 'ca609', 'ca610', 'ca611', 'ca612', 'ca613', 'ca614', 'ca615', 'ca616', 'ca617', 'ca618', 'ca619', 'ca620', 'ca621', 'ca622', 'ca623', 'ca624', 'ca625', 'ca626', 'ca627', 'ca628', 'ca629', 'ca630', 'ca631', 'ca632', 'ca633', 'ca634', 'ca635', 'ca636', 'ca637', 'ca638', 'ca639', 'ca640', 'ca641', 'ca642', 'ca643', 'ca644', 'ca645', 'ca646', 'ca647', 'ca648', 'ca649', 'ca650', 'ca651', 'ca652', 'ca653', 'ca654', 'ca655', 'ca656', 'ca657', 'ca658', 'ca659', 'ca660', 'ca661', 'ca662', 'ca663', 'ca664', 'ca665', 'ca666', 'ca667', 'ca668', 'ca669', 'ca670', 'ca671', 'ca672', 'ca673', 'ca674', 'ca675', 'ca676', 'ca677', 'ca678', 'ca679', 'ca680', 'ca681', 'ca682', 'ca683', 'ca684', 'ca685', 'ca686', 'ca687', 'ca688', 'ca689', 'ca690', 'ca691', 'ca692', 'ca693', 'ca694', 'ca695', 'ca696', 'ca697', 'ca698', 'ca699', 'ca700', 'ca701', 'ca702', 'ca703', 'ca704', 'ca705', 'ca706', 'ca707', 'ca708', 'ca709', 'ca710', 'ca711', 'ca712', 'ca713', 'ca714', 'ca715', 'ca716', 'ca717', 'ca718', 'ca719', 'ca720', 'ca721', 'ca722', 'ca723', 'ca724', 'ca725', 'ca726', 'ca727', 'ca728', 'ca729', 'ca730', 'ca731', 'ca732', 'ca733', 'ca734', 'ca735', 'ca736', 'ca737', 'ca738', 'ca739', 'ca740', 'ca741', 'ca742', 'ca743', 'ca744', 'ca745', 'ca746', 'ca747', 'ca748', 'ca749', 'ca750', 'ca751', 'ca752', 'ca753', 'ca754', 'ca755', 'ca756', 'ca757', 'ca758', 'ca759', 'ca760', 'ca761', 'ca762', 'ca763', 'ca764', 'ca765', 'ca766', 'ca767', 'ca768', 'ca769', 'ca770', 'ca771', 'ca772', 'ca773', 'ca774', 'ca775', 'ca776', 'ca777', 'ca778', 'ca779', 'ca780', 'ca781', 'ca782', 'ca783', 'ca784', 'ca785', 'ca786', 'ca787', 'ca788', 'ca789', 'ca790', 'ca791', 'ca792', 'ca793', 'ca794', 'ca795', 'ca796', 'ca797', 'ca798', 'ca799', 'ca800', 'ca801', 'ca802', 'ca803', 'ca804', 'ca805', 'ca806', 'ca807', 'ca808', 'ca809', 'ca810', 'ca811', 'ca812', 'ca813', 'ca814', 'ca815', 'ca816', 'ca817', 'ca818', 'ca819', 'ca820', 'ca821', 'ca822', 'ca823', 'ca824', 'ca825', 'ca826', 'ca827', 'ca828', 'ca829', 'ca830', 'ca831', 'ca832', 'ca833', 'ca834', 'ca835', 'ca836', 'ca837', 'ca838', 'ca839', 'ca840', 'ca841', 'ca842', 'ca843', 'ca844', 'ca845', 'ca846', 'ca847', 'ca848', 'ca849', 'ca850', 'ca851', 'ca852', 'ca853', 'ca854', 'ca855', 'ca856', 'ca857', 'ca858', 'ca859', 'ca860', 'ca861', 'ca862', 'ca863', 'ca864', 'ca865', 'ca866', 'ca867', 'ca868', 'ca869', 'ca870', 'ca871', 'ca872', 'ca873', 'ca874', 'ca875', 'ca876', 'ca877', 'ca878', 'ca879', 'ca880', 'ca881', 'ca882', 'ca883', 'ca884', 'ca885', 'ca886', 'ca887', 'ca888', 'ca889', 'ca890', 'ca891', 'ca892', 'ca893', 'ca894', 'ca895', 'ca896', 'ca897', 'ca898', 'ca899', 'ca900', 'ca901', 'ca902', 'ca903', 'ca904', 'ca905', 'ca906', 'ca907', 'ca908', 'ca909', 'ca910', 'ca911', 'ca912', 'ca913', 'ca914', 'ca915', 'ca916', 'ca917', 'ca918', 'ca919', 'ca920', 'ca921', 'ca922', 'ca923', 'ca924', 'ca925', 'ca926', 'ca927', 'ca928', 'ca929', 'ca930', 'ca931', 'ca932', 'ca933', 'ca934', 'ca935', 'ca936', 'ca937', 'ca938', 'ca939', 'ca940', 'ca941', 'ca942', 'ca943', 'ca944', 'ca945', 'ca946', 'ca947', 'ca948', 'ca949', 'ca950', 'ca951', 'ca952', 'ca953', 'ca954', 'ca955', 'ca956', 'ca957', 'ca958', 'ca959', 'ca960', 'ca961', 'ca962', 'ca963', 'ca964', 'ca965', 'ca966', 'ca967', 'ca968', 'ca969', 'ca970', 'ca971', 'ca972', 'ca973', 'ca974', 'ca975', 'ca976', 'ca977', 'ca978', 'ca979', 'ca980', 'ca981', 'ca982', 'ca983', 'ca984', 'ca985', 'ca986', 'ca987', 'ca988', 'ca989', 'ca990', 'ca991', 'ca992', 'ca993', 'ca994', 'ca995', 'ca996', 'ca997', 'ca998', 'ca999', 'ca1000']

ca01 has following words:
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', ...]

ca01 has 2242 words

Categories or files in brown corpus:
['adventure', 'belles_lettres', 'editorial', 'fiction', 'government', 'hobbies', 'humor', 'learned', 'lore', 'mystery', 'news', 'religion', 'reviews', 'romance', 'science_fiction']

Statistics for each text:
AvgWordLen AvgSentenceLen No.ofTimesEachWordAppearsOnAvg FileName
3 22 2 ca01
8 23 2 ca02
8 20 2 ca03
9 25 2 ca04
8 26 3 ca05
8 22 2 ca06
9 18 2 ca07
8 21 2 ca08
9 19 2 ca09
8 21 2 ca10
8 22 2 ca11
8 22 2 ca12
8 20 2 ca13
8 17 2 ca14
8 21 2 ca15
8 20 2 ca16
8 22 2 ca17
8 22 2 ca18
8 20 2 ca19
8 23 2 ca20
9 20 2 ca21
8 21 2 ca22
8 20 2 ca23
8 22 2 ca24
8 21 2 ca25
8 21 2 ca26
8 19 2 ca27
```

Conclusion:

Various corpora like Brown, Inaugural, Reuters, and UDHR were studied using different methods (fields, raw, words, sentences, categories), helping to understand text structure and usage in NLP.

Practical 2

Aim(B): Create and use your own corpora (plaintext, categorical).

Theory:

A corpus (plural: corpora) is a large and organized collection of text data used in Natural Language Processing (NLP).

It can include books, articles, speeches, or any written content stored in digital form. Corpora are used to analyze language patterns, study word usage, and train machine learning models.

Using tools like NLTK, we can process corpora to extract words, sentences, and other linguistic information.

In this practical, we create and use our own corpus using NLTK. First, text data is collected and stored in text files to form a plaintext corpus.

Then, the data is organized into different folders based on topics to create a categorical corpus.

The corpus is loaded using NLTK tools, and various methods like `raw()`, `words()`, `sents()`, `fileids()`, and `categories()` are used to analyze the text.

This practical helps in understanding the structure, content, and patterns of text data and is useful for tasks like text classification and language analysis.

Source code:

```
import nltk

from nltk.corpus import PlaintextCorpusReader

# Correct path (must be in ONE line)

corpus_root = r'C:\Users\Tarique\Desktop\SEMESTER-4\NATURAL LANGUAGE
PROCESSING (NLP)\NLP\PRACTICAL 2'

# Load corpus

filelist = PlaintextCorpusReader(corpus_root, '.*')

print('\nFile list:\n')

print(filelist.fileids())

print(filelist.root)

# Statistics

print("\n\nStatistics for each text:\n")
```

```
print('AvgWordLen\tAvgSentenceLen\tAvgWordFreq\tFileName')

for fileid in filelist.fileids():

    num_chars = len(filelist.raw(fileid))

    num_words = len(filelist.words(fileid))

    num_sents = len(filelist.sents(fileid))

    num_vocab = len(set([w.lower() for w in filelist.words(fileid)]))

    print(

        int(num_chars / num_words), '\t\t',

        int(num_words / num_sents), '\t\t',

        int(num_words / num_vocab), '\t\t',

        fileid

    )
```

Output:

```
File list:

['Harshad Patil 22306A1020.txt', 'NLP PRACT 2 B is almost done.txt']
D:\MSCIT\SEM 4\2 Natural Language Processing\Practicals\Practical 2

Statistics for each text:

AvgWordLen      AvgSentenceLen  no.ofTimesEachWordAppearsOnAvg  FileName
4                7                1                Harshad Patil 22306A1020.txt
4                3                3                NLP PRACT 2 B is almost done.txt
```

Conclusion:

In this practical, we successfully created and used our own corpora in plaintext and categorical formats. This helps in organizing and analyzing text data efficiently, which is essential for performing various NLP tasks like classification and text processing.

Practical 2

Aim(C): Study conditional frequency distribution.

Theory:

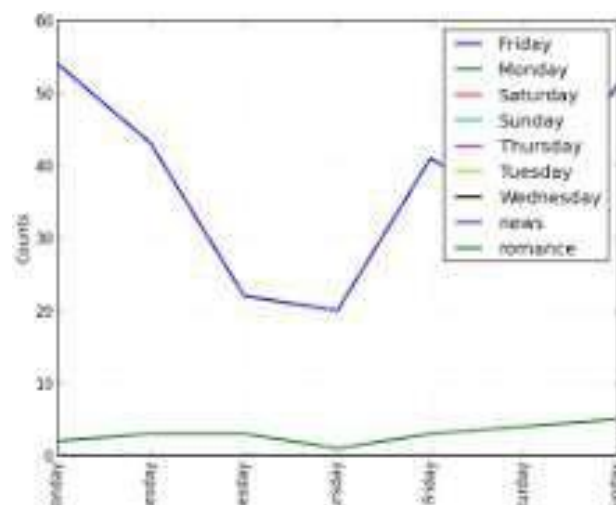
A Conditional Frequency Distribution (CFD) shows how often a feature (like a word) occurs under a specific condition or category.

In simple terms, it tells us:

“How frequently something appears in different groups.”

Example

- Condition → Category (e.g., news, sports, politics)
- Event → Words (e.g., *economy*, *game*, *election*)
 1. CFD helps analyze word distribution across categories
 2. It is useful for understanding patterns in text data
 3. Widely used in NLP tasks like classification and analysis



- This graph represents a Conditional Frequency Distribution (CFD).
- The x-axis shows categories (like days or genres), and the y-axis shows the frequency (count) of words.
- Each colored line represents a different condition (e.g., Friday, Monday, romance).
- The graph helps compare how frequently words appear across different conditions.

Source Code:

```
# Process a sequence of pairs
text = ['The', 'Fulton', 'County', 'Grand', 'Jury', 'said',
...]
pairs = [('news', 'The'), ('news', 'Fulton'), ('news',
```

```

'County'), ...]
import nltk
from nltk.corpus import brown
fd = nltk.ConditionalFreqDist(
    (genre, word)
    for genre in brown.categories()
    for word in brown.words(categories=genre)
)
genre_word = [
    (genre, word)
    for genre in ['news', 'romance']
    for word in brown.words(categories=genre)
]
print(len(genre_word))
print(genre_word[:4])
print(genre_word[-4:])
cfd = nltk.ConditionalFreqDist(genre_word)
print(cfd)
print(cfd.conditions())
print(cfd['news'])
print(cfd['romance'])
print(list(cfd['romance']))
from nltk.corpus import inaugural
cfd = nltk.ConditionalFreqDist(
    (target, fileid[:4])
    for fileid in inaugural.fileids()
    for w in inaugural.words(fileid)
    for target in ['america', 'citizen']
    if w.lower().startswith(target)
)
from nltk.corpus import udhr
languages = [
    'Chickasaw', 'English', 'German_Deutsch',
    'Greenlandic_Inuktitut', 'Hungarian_Magyar',
    'Ibibio_Efik'
]
cfd = nltk.ConditionalFreqDist(
    (lang, len(word))
    for lang in languages
    for word in udhr.words(lang + '-Latin1')
)
cfd.tabulate(conditions=['English', 'German_Deutsch'],
    samples=range(10), cumulative=True)

```

Output:

```

170576
[('news', 'The'), ('news', 'Fulton'), ('news', 'County'), ('news', 'Grand')]
[('romance', 'afraid'), ('romance', 'not'), ('romance', ''), ('romance', '.')]
<ConditionalFreqDist with 2 conditions>
['news', 'romance']
<FreqDist with 14394 samples and 100554 outcomes>
<FreqDist with 8452 samples and 70022 outcomes>
['', '.', 'the', 'and', 'to', 'a', 'of', '``', ''''', 'was', 'I', 'in', 'he', 'had', '?',
t', 'He', 'on', 'him', 'said', '!', '--', 'be', 'as', ';', 'have', 'but', 'not', 'would'

```

Conclusion:

In this practical, we studied conditional frequency distribution, which helps analyze the frequency of words under different conditions. This is useful for understanding patterns in text data and improving NLP tasks like text analysis and classification.

Practical 2

Aim(D): Study of tagged corpora with methods like `tagged_sents`, `tagged_words`.

Theory:

A tagged corpus is a collection of text where each word is associated with a tag, usually a Part-of-Speech (POS) tag such as noun, verb, adjective, etc.

Example:

The/DT dog/NN barks/VBZ

1. Words are stored along with their grammatical tags
2. Helps in understanding sentence structure and grammar
3. Used in NLP tasks like POS tagging, parsing, and machine learning
 - Tagged corpora provide word + grammatical information
 - Methods like `tagged_words()` and `tagged_sents()` help in analyzing sentence structure
 - Important for many NLP applications

Uses of Tagged Corpora

- Part-of-Speech (POS) tagging
- Grammar analysis
- Machine translation
- Text processing

Source Code:

```
import nltk

from nltk import tokenize

nltk.download('punkt')

nltk.download('words')

para = "Hello! My name is Harshad Patil. Today we will be learning NLTK."

sents = tokenize.sent_tokenize(para)

print("\nsentence tokenization\n=====\\n", sents)

# word tokenization

print("\nword tokenization\n=====\\n")
```

```
for index in range(len(sents)):  
    words = tokenize.word_tokenize(sents[index])  
    print(words)
```

Output:

```
[nltk_data] Downloading package punkt to  
[nltk_data] C:\Users\Alfiya\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping tokenizers\punkt.zip.  
[nltk_data] Downloading package words to  
[nltk_data] C:\Users\Alfiya\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping corpora\words.zip.  
  
sentence tokenization  
=====
```

```
['Hello!', 'My name is Harshad Patil.', 'Today we will be learning NLTK.']  
  
word tokenization  
=====
```

```
['Hello', '!']  
['My', 'name', 'is', 'Harshad', 'Patil', '.']  
['Today', 'we', 'will', 'be', 'learning', 'NLTK', '.']
```

Conclusion:

In this practical, we studied tagged corpora using methods like `tagged_sents()` and `tagged_words()`. This helps in understanding the grammatical structure of text, which is important for NLP tasks such as POS tagging and language analysis.

Practical 2

Aim(E): Write a program to find the most frequent noun tags.

Theory:

In Natural Language Processing (NLP), Part-of-Speech (POS) tagging is used to assign grammatical categories like noun, verb, adjective, etc., to each word in a sentence.

Noun tags include:

- NN → Singular common noun (e.g., book, student)
- NNS → Plural noun (e.g., books, students)
- NNP → Proper noun singular (e.g., India, Mumbai)
- NNPS → Proper noun plural (e.g., Indians)

Source code:

```
import nltk

from collections import defaultdict

text = nltk.word_tokenize("Harshad likes to play cricket.
Harshad does not like to play with hearts.")

tagged = nltk.pos_tag(text)

print(tagged)

addNounWords = [] # Checking if it is a noun or not

count = 0

for words in tagged:
    val = tagged[count][1]

    if val in ('NN', 'NNS', 'NNPS', 'NNP'):
        addNounWords.append(tagged[count][0])

    count += 1

print(addNounWords)

temp = defaultdict(int)

for sub in addNounWords:
    # Memoizing count
    for wrd in sub.split():
```

```
temp[wrđ] += 1

# Getting max frequency

res = max(temp, key=temp.get)

# Printing result

print("Word with maximum frequency: " + str(res))
```

Output:

```
Python 3.8.6rc1 Shell
File Edit Shell Debug Options Window Help
Python 3.8.6rc1 (tags/v3.8.6rc1:08bd63d, Sep 7 2020, 23:10:23) [MSC
v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more informat
ion.
>>>
===== RESTART: C:/Users/admin/Documents/NLP 2E.py =====
=====
[('Harshad', 'NNP'), ('likes', 'NNS'), ('to', 'TO'), ('play', 'VB'),
('cricket', 'NN'), ('.', '.'), ('Harshad', 'NNP'), ('does', 'VBZ'),
('not', 'RB'), ('like', 'VB'), ('to', 'TO'), ('play', 'VB'), ('with'
, 'IN'), ('hearts', 'NNS'), ('.', '.')]
['Harshad', 'likes', 'cricket', 'Harshad', 'hearts']
Word with maximum frequency: Harshad
>>>
```

Conclusion:

In this practical, we successfully identified the most frequent noun tags from the given text. This helps in understanding the distribution of nouns in a corpus and is useful for text analysis and NLP applications.

Practical 2

Aim(F): Map words to the properties using Python Dictionaries.

Theory:

Mapping words to properties means storing a word and its related information (properties) together using a dictionary.

In Python, a dictionary works with key–value pairs:

- Key (word) → the main item
- Value (properties) → information related to that word

In Natural Language Processing, mapping words to properties can mean:

- Word → POS tag
- Word → meaning
- Word → category

Source code:

```
#creating and printing a dictionary by mapping word with its
properties

thisdict = {

    "brand": "Ford",

    "model": "Mustang",

    "year": 1964

}

print(thisdict)

print(thisdict["brand"])

print(len(thisdict))

print(type(thisdict))
```

Output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
Ford
3
<class 'dict'>
```

Conclusion: In this practical, we successfully mapped words to their properties using Python dictionaries.

Practical 2

Aim(G): Study: i)Default Tagger, ii)Regular expression tagger, iii)UnigramTagger

Theory:

i)Default Tagger

A Default Tagger assigns the same POS tag to every word in a text.

- It is the simplest tagger.
- It does not analyze the word; it just gives a fixed tag (like NN for noun) to all words.
- Used as a baseline or fallback when no other tagger can decide.

ii) Regular Expression Tagger (RegexpTagger)

A RegexpTagger assigns tags based on pattern matching rules (regular expressions).

- It checks the form of words (like ending with "ing", "ed").
- Tags are assigned using predefined patterns.
- More accurate than Default Tagger.

iii) Unigram Tagger

A Unigram Tagger assigns tags based on frequency of words learned from training data.

- It looks at each word individually (uni = one).
- It assigns the most frequent tag for that word from a trained corpus.
- Requires training data.

Source Code:

i)Default Tagger

```
import nltk
nltk.download('treebank')
import nltk
from nltk.tag import DefaultTagger
from nltk.corpus import treebank
# Create Default Tagger (assigns 'NN' to every word)
exptagger = DefaultTagger('NN')
# Load test sentences
testsentences = treebank.tagged_sents()[1000:]
# Evaluate accuracy (correct method)
```



```
print("Accuracy:", exptagger.accuracy(testsentences))  
# Tagging new sentences  
sentences = [['Hi', ',', ''], ['How', 'are', 'you', '?']]  
print("Tagged Sentences:", exptagger.tag_sents(sentences))
```

Output:

```
[nltk_data] Downloading package treebank to  
[nltk_data] C:\Users\Alfiya\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping corpora\treebank.zip.  
True
```

Accuracy: 0.13198749536374715

Tagged Sentences: [['Hi', 'NN'), (',', 'NN')], [['How', 'NN'), ('are', 'NN'), ('you', 'NN'), ('?', 'NN')]]

ii) Reg Regular Expression Tagger (RegexTagger)

```
from nltk.corpus import brown  
from nltk.tag import RegexTagger  
test_sent = brown.sents(categories='news')[0]  
regex_tagger = RegexTagger(  
    [(r'^-?[0-9]+(\.[0-9]+)?$', 'CD'), # cardinal numbers  
     (r'(The|the|A|a|An|an)$', 'AT'), # articles  
     (r'.*able$', 'JJ'), # adjectives  
     (r'.*ness$', 'NN'), # nouns formed from adjectives  
     (r'.*ly$', 'RB'), # adverbs  
     (r'.*s$', 'NNS'), # plural nouns  
     (r'.*ing$', 'VBG'), # gerunds  
     (r'.*ed$', 'VBD'), # past tense verbs  
     (r'.*', 'NN') # nouns (default)  
])  
print(regex_tagger)  
print(regex_tagger.tag(test_sent))
```

Output:

```
<Regex Tagger: size=9>
[('The', 'AT'), ('Fulton', 'NN'), ('County', 'NN'), ('Grand', 'NN'), ('Jury', 'NN'), ('said', 'NN'), ('Friday', 'NN'), ('an', 'NN'), ('of', 'NN'), ('Atlanta's', 'NNS'), ('recent', 'NN'), ('primary', 'NN'), ('election', 'NN'), ('produced', 'VBD'), ('' ('evidence', 'NN'), ('''', 'NN'), ('that', 'NN'), ('any', 'NN'), ('irregularities', 'NNS'), ('took', 'NN'), ('place', 'NN'),
```

iii) Unigram tagger

```
from nltk.tag import UnigramTagger
from nltk.corpus import treebank
train_sents = treebank.tagged_sents()[1:10]
tagger = UnigramTagger(train_sents) # Initializing
print(treebank.sents()[0])
print('\n', tagger.tag(treebank.sents()[0]))
tagger.tag(treebank.sents()[0])
tagger = UnigramTagger(model = {'Pierre': 'NN'})
#Overriding the context model
print('\n', tagger.tag(treebank.sents()[0]))
```

Output:

```
[('Pierre', 'NNP'), ('Vinken', 'NNP'), (',', ','), ('61', 'CD'), ('years', 'NNS'), ('old', 'JJ'), (',', ','), ('', ''), ('DT'), ('board', 'NN'), ('as', 'IN'), ('a', 'DT'), ('nonexecutive', 'JJ'), ('director', 'NN'), ('Nov.', 'NNP'),

[('Pierre', 'NN'), ('Vinken', None), (',', None), ('61', None), ('years', None), ('old', None), (',', None), ('one), ('board', None), ('as', None), ('a', None), ('nonexecutive', None), ('director', None), ('Nov.', None), (':
```

Conclusion:

Default Tagger is simple but least accurate, Regular Expression Tagger improves tagging using patterns, and Unigram Tagger gives better accuracy using trained data.

Practical 2

Aim(H): Find different words from a given plaintext without any spaces by comparing this text with a given corpus of words. Also find the score of words.

Theory:

This practical focuses on identifying meaningful words from a continuous text (without spaces) by comparing it with a given corpus (dictionary of valid words).

The program:

- Takes a string with no spaces (e.g., ilovemumbai)
- Compares it with a corpus of known words
- Breaks the string into valid words found in the corpus
- Assigns a score to each word based on criteria like frequency, length, or probability

Source Code:

```
#creating and printing a dictionary by mapping word with its properties
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print(thisdict)
print(thisdict["brand"])
print(len(thisdict))
print(type(thisdict))
```

Output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
Ford
3
<class 'dict'>
```

Conclusion:

This task demonstrates how to identify meaningful words from a continuous plaintext without spaces by comparing it with a corpus. It also shows how scoring helps select the most probable words, improving word segmentation and accuracy.

Practical 3

Aim(A): Study of Wordnet Dictionary with methods as synsets, definitions, examples, antonyms

Theory:

WordNet is a large lexical database of English available in NLTK. It groups words into sets of synonyms called synsets and provides semantic relationships like meanings and opposites.

Synsets

- A synset is a group of words with similar meaning.
- Example: *car*, *automobile* belong to the same synset.

Definitions

- Gives the meaning of a word.

Source Code:

```
import nltk

from nltk.corpus import wordnet

nltk.download('wordnet')

print(wordnet.synsets("sunrise"))

print("My word is Sunrise:- \n", "Definition:",
wordnet.synset("sunrise.n.01").definition())

print("Examples:",
wordnet.synset("sunrise.n.01").examples())

anto = wordnet.lemma('sunrise.n.01.sunrise')

print("\nAntonym of word Sell (Noun):", anto.antonyms())
```

Output:

```
[Synset('dawn.n.01'), Synset('sunrise.n.02'), Synset('sunrise.n.03'), Synset('sunrise.s.01')]
My word is Sunrise:-
Definition: the first light of day
Examples: ['we got up before dawn', 'they talked until morning']

Antonym of word Sell (Noun): [Lemma('sunset.n.01.sunset')]
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\Admin\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

Conclusion:

The study of WordNet demonstrates how words are organized into synsets and connected through relationships like definitions, examples, and antonyms. It helps in understanding word meanings, semantic relationships, and improves natural language processing tasks

Practical 3

Aim(B): Study lemmas, hyponyms, hypernyms, entailments

Theory:

WordNet organizes words based on their meanings and relationships. These relationships help in understanding how words are connected semantically.

Lemmas

- Lemmas are the base forms of words in a synset.
- They represent different words with the same meaning.
Example: *car*, *automobile*

Hyponyms

- Hyponyms are more specific words under a general term.
Example: *car*, *bike* are hyponyms of *vehicle*

Hypernyms

- Hypernyms are more general words (parent category).
Example: *vehicle* is a hypernym of *car*

Entailments

- Entailments show actions that are logically included in another action (mostly verbs).
Example: *snore* entails *sleep*

Source Code:

```
import nltk

from nltk.corpus import wordnet

print(wordnet.synsets("computer"))

print(wordnet.synset("computer.n.01").lemma_names())

for e in wordnet.synsets("computer"):
    print(f'{e} --> {e.lemma_names()}')
    print(wordnet.synset('computer.n.01').lemmas())
    print(wordnet.lemma('computer.n.01.computing_device').synset())
    print(wordnet.lemma('computer.n.01.computing_device').name())
    print("\n\n Hyponyms of computer:\n")
    syn = wordnet.synset('computer.n.01')
    print(syn.hyponyms)

print([lemma.name() for synset in syn.hyponyms() for lemma in synset.lemmas()])
```

```

print("\n\n Hyponyms of vehicle:\n")

vehicle = wordnet.synset('vehicle.n.01')

print(vehicle.hyponyms)

print([lemma.name() for synset in vehicle.hyponyms() for
lemma in synset.lemmas()])

print("\n\n Hyponyms of car:\n")

car = wordnet.synset('car.n.01')

print(car.hyponyms)

print([lemma.name() for synset in car.hyponyms() for lemma
in synset.lemmas()])

print(car.lowest_common_hypernyms(vehicle))

print("\n\nVehil")

vehil=wordnet.synset('vehicle.n.01')

print(vehil.hypernyms)

print([lemma.name() for synset in vehil.hypernyms() for
lemma in synset.lemmas()])

```

Output:

```

Synset('computer.n.01') --> ['computer', 'computing_machine', 'computing_device', '
em']
[[Lemma('computer.n.01.computer'), Lemma('computer.n.01.computing_machine'), Lemma('
r'), Lemma('computer.n.01.electronic_computer'), Lemma('computer.n.01.information_p
Synset('computer.n.01')
computing_device

Hyponyms of computer:
<bound method _WordNetObject.hyponyms of Synset('computer.n.01')>
Synset('calculator.n.01') --> ['calculator', 'reckoner', 'figurer', 'estimator', 'c
[[Lemma('computer.n.01.computer'), Lemma('computer.n.01.computing_machine'), Lemma('
r'), Lemma('computer.n.01.electronic_computer'), Lemma('computer.n.01.information_p
Synset('computer.n.01')
computing_device

Hyponyms of computer:
<bound method _WordNetObject.hyponyms of Synset('computer.n.01')>

Hyponyms of vehicle:
['mode', 'client', 'guest', 'digital_computer', 'predictor', 'web_kite', 'website', 'Internet_sit
ogus_computer', 'Turing_machine', 'number_cruncher', 'home_computer', 'papi-mutual_machine', 'bot

Hyponyms of vehicle:
<bound method _WordNetObject.hyponyms of Synset('vehicle.n.01')>
['skitob', 'sled', 'sledge', 'sleigh', 'military_vehicle', 'wheeled_vehicle', 'bumper_car', 'Dog
le', 'craft']

Hyponyms of car:
<bound method _WordNetObject.hyponyms of Synset('car.n.01')>
['sport_utility', 'sport_utility_vehicle', 'S.U.V.', 'SUV', 'horseless_carriage', 'ambulance', 'r
gunster', 'subcompact', 'subcompact_car', 'touring_car', 'phaeton', 'tourer', 'beach_wagon', '
station_wagon', 'wagon', 'coupe', 'pace_car', 'Stanley_Steamer', 'electric', 'electric_automob
minicar', 'compact', 'compact_car', 'hot_rod', 'hot-rod', 'cruiser', 'police_cruiser', 'petrol_
e', 'sedan', 'saloon', 'sports_car', 'sport_car', 'stock_car', 'cab', 'hack', 'taxi', 'taxicab',
T', 'minivan', 'limousine', 'limo', 'used-car', 'secondhand_car', 'bus', 'jalopy', 'heav']
[Synset('vehicle.n.01')]

Vehil:
<bound method _WordNetObject.hypernyms of Synset('vehicle.n.01')>
['conveyance', 'transport']

```

Conclusion:

The study of lemmas, hyponyms, hypernyms, and entailments helps in understanding relationships between words and their meanings. It improves semantic analysis and enhances language understanding in NLP.

Practical 3

Aim(C): Write a program using python to find synonym and antonym of word "active" using Wordnet.

Theory:

Finding synonyms and antonyms using WordNet means using the WordNet lexical database to identify words that have similar meanings (synonyms) or opposite meanings (antonyms) to a given word. WordNet organizes words into sets of synonyms called synsets, where each synset represents a specific meaning of a word and provides related words with the same sense.

By exploring these synsets, we can extract synonyms of a word, which are words that can be used interchangeably in certain contexts. Similarly, WordNet stores relationships that point to antonyms for some words, allowing us to find words with opposite meanings.

This process is widely used in Natural Language Processing for tasks such as text analysis, language understanding, sentiment analysis, and improving search or recommendation systems.

Source Code:

```
from nltk.corpus import wordnet
print( wordnet.synsets("active"))

print("\n", wordnet.lemma('active.a.01.active').antonyms())
```

Output:

```
[Synset('active_agent.n.01'), Synset('active_voice.n.01'), Synset('active.n.03'), Synset(
Synset('active.s.04'), Synset('active.a.05'), Synset('active.a.06'), Synset('active.a.07
e.a.10'), Synset('active.a.11'), Synset('active.a.12'), Synset('active.a.13'), Synset('a
[Lemma('inactive.a.02.inactive')]
```

Conclusion:

This program demonstrates how to use WordNet in Python to find synonyms and antonyms of the word “active.” It highlights the importance of lexical databases in understanding word meanings and relationships, which is useful for various NLP applications.

Practical 3

Aim(D): Compare two nouns.

Theory:

Comparing two nouns means analyzing their semantic similarity or relatedness in terms of meaning.

In Natural Language Processing, this is often done using lexical databases like WordNet, which organizes nouns into hierarchies based on their meanings (hypernyms, hyponyms, synonyms).

By comparing two nouns, we can determine how closely they are related—for example, “dog” and “wolf” are semantically closer than “dog” and “car” because they share more common characteristics in the lexical hierarchy.

Source Code:

```
import nltk

from nltk.corpus import wordnet

# Get synsets
syn1 = wordnet.synsets('football')
syn2 = wordnet.synsets('soccer')

# A word may have multiple synsets, so compare each synset
for s1 in syn1:
    for s2 in syn2:
        print("Path similarity of:")
        print(s1, '(', s1.pos(), ')', '[', s1.definition(), ']')
        print(s2, '(', s2.pos(), ')', '[', s2.definition(), ']')
        print("    is", s1.path_similarity(s2))
        print()
```

Output:

```
Path similarity of:
Synset('football.n.01') ( n ) [ any of various games played with a ball (
each other's goal ]
Synset('soccer.n.01') ( n ) [ a football game in which two teams of 11 p:
is 0.5

Path similarity of:
Synset('football.n.02') ( n ) [ the inflated oblong ball used in playing
Synset('soccer.n.01') ( n ) [ a football game in which two teams of 11 p:
is 0.05
```

Conclusion:

Comparing two nouns helps in understanding their similarities and differences in meaning, usage, and context, improving semantic analysis in natural language processing.

Practical 3

Aim(E): Handling Stopwords

1. **Using nltk Adding or Removing Stop Words in NLTK's Default Stop Word List**
2. **Using Gensim, add or remove stop words in Default Gensim stop words List.**
3. **Using SpaCy, , add or remove Stop Words in Default SpaCy stop words List.**

Theory:

Handling stopwords is an important step in natural language processing (NLP) to remove commonly used words that do not carry significant meaning, such as “*is*,” “*the*,” “*and*”. Removing stopwords helps reduce noise and improves the efficiency of text analysis, search, and machine learning models.

1. **Using NLTK:** NLTK provides a default list of stopwords for multiple languages. Users can add or remove words from this list according to the needs of their application. For example, you might add domain-specific words like “*please*” or remove words like “*not*” if negation is important for sentiment analysis. This customization allows more precise text preprocessing.
2. **Using Gensim:** Gensim also provides a default stopwords list that can be modified. You can add custom stopwords by extending the set, or remove specific words from the list to retain important tokens during text processing. This is useful when preparing text for topic modeling or similarity analysis.
3. **Using SpaCy:** SpaCy includes its own default stopwords set. Users can add words to the stopwords list using `nlp.Defaults.stop_words.add("word")` or remove words using `nlp.Defaults.stop_words.remove("word")`. This flexibility allows developers to handle text in specialized domains where certain words may need to be kept or ignored.

Source Code:

1. Using nltk Adding or Removing Stop Words in NLTK's Default Stop Word List

```
import nltk

from nltk.corpus import stopwords

nltk.download('punkt')

nltk.download('stopwords')

from nltk.tokenize import word_tokenize

text = "the quick brown fox jumps over the lazy dog but not a cat"

text_tokens = word_tokenize(text)
```

```
tokens_without_sw = [word for word in text_tokens if not
word in stopwords.words()]
print(tokens_without_sw)

all_stopwords = stopwords.words('english')
all_stopwords.append('play')

text_tokens = word_tokenize(text)

tokens_without_sw = [word for word in text_tokens if not
word in all_stopwords]
print(tokens_without_sw)

all_stopwords.remove('not')

text_tokens = word_tokenize(text)

tokens_without_sw = [word for word in text_tokens if not
word in all_stopwords]
print(tokens_without_sw)
```

Output:

```
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\Admin\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   C:\Users\Admin\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
['quick', 'brown', 'fox', 'jumps', 'lazy']
['quick', 'brown', 'fox', 'jumps', 'lazy', 'dog', 'cat']
['quick', 'brown', 'fox', 'jumps', 'lazy', 'dog', 'not', 'cat']
```

2. Using Gensim, add or remove stop words in Default Gensim stop words List.

```
!pip install genism

#Using Gensim, add or remove stop words in Default Gensim stop words
List.

# pip install gensim nltk

import gensim

from gensim.parsing.preprocessing import remove_stopwords, STOPWORDS

import nltk

from nltk.tokenize import word_tokenize

# Ensure necessary NLTK data is downloaded
```

```
import nltk

import gensim

from gensim.parsing.preprocessing import remove_stopwords, STOPWORDS

from nltk.tokenize import word_tokenize

# Download required resources

nltk.download('punkt')

nltk.download('stopwords')

text = "Harshad likes to play cricket, however he is not too fond of
basketball."

# Remove stopwords using gensim default

filtered_sentence = remove_stopwords(text)

print("After remove_stopwords():")

print(filtered_sentence)

# Print default stopwords

all_stopwords = STOPWORDS

print("\nDefault STOPWORDS:")

print(list(all_stopwords)[:20])    # print first 20 only

# Add custom stopwords

# -----

all_stopwords_gensim = STOPWORDS.union(set(['likes', 'play']))

text_tokens = word_tokenize(text)

tokens_without_sw = [

    word for word in text_tokens if word.lower() not in
    all_stopwords_gensim

]

print("\nAfter adding 'likes' and 'play' to stopwords:")

print(tokens_without_sw)

sw_list = {"not"}

all_stopwords_gensim = STOPWORDS.difference(sw_list)

text_tokens = word_tokenize(text)

tokens_without_sw = [
```

```

        word for word in text_tokens if word.lower() not in
all_stopwords_gensim

]

print("\nAfter removing 'not' from stopwords:")

print(tokens_without_sw)

```

Output:

```

After remove_stopwords():
Harshad likes play cricket, fond basketball.

Default STOPWORDS:
['see', 'from', 'amongst', 'almost', 'could', 'perhaps', 'on', 'hereby', 'amount',
asnt', 'hereafter', 'somehow', 'used']

After adding 'likes' and 'play' to stopwords:
['Harshad', 'cricket', ',', 'fond', 'basketball', '.']

After removing 'not' from stopwords:
['Harshad', 'likes', 'play', 'cricket', ',', 'not', 'fond', 'basketball', '.']

[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\Admin\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!

```

3. Using SpaCy, add or remove Stop Words in Default SpaCy stop words List.

```

!pip install spacy

!python -m spacy download en_core_web_sm

#Using SpaCy, , add or remove Stop Words in Default SpaCy stop words
List.

#pip install spacy

#python -m spacy download en_core_web_sm

#python -m spacy download en

import spacy

import nltk

from nltk.tokenize import word_tokenize

sp = spacy.load('en_core_web_sm')

#add the word play to the NLTK stop word collection

all_stopwords = sp.Defaults.stop_words

all_stopwords.add("play")

```

```
text = "Harshad likes to play cricket, however he is not too fond of
basketball."

text_tokens = word_tokenize(text)

tokens_without_sw = [word for word in text_tokens if not
word in all_stopwords]

print(tokens_without_sw)

#remove 'not' from stop word collection
all_stopwords.discard('not')

tokens_without_sw = [word for word in text_tokens if not
word in all_stopwords]

print(tokens_without_sw)
```

Output:

```
['Harshad', 'likes', 'cricket', ',', 'not', 'fond', 'basketball', '.']
['Harshad', 'likes', 'cricket', ',', 'not', 'fond', 'basketball', '.']
```

Conclusion:

Handling stopwords using NLTK, Gensim, and SpaCy shows how unwanted words can be customized by adding or removing them from default lists. This improves text preprocessing and enhances the performance of NLP tasks.

Practical 4

Aim(A): Tokenization using Python's split() function.

Theory:

Tokenization using Python's split() function is the process of breaking a text into smaller units called tokens, usually words or sentences, using the built-in string method split().

By default, split() divides a string wherever there is a space, producing a list of words. For example, the sentence "*I love NLP*" can be tokenized using split() to get ['I', 'love', 'NLP']. Tokenization is a fundamental step in natural language processing (NLP) because it converts raw text into a structured format that can be further processed for tasks like text analysis, sentiment analysis, and machine learning.

While split() is simple and easy to use, it does not handle punctuation, special characters, or complex tokenization rules, so more advanced libraries like NLTK, SpaCy, or Gensim are often used for robust tokenization.

Source Code:

```
text = """ This tool is in a beta stage. Electric cars are running petrol
cars raw and rough experience. It also supports custom battery model,
prebuilt charging model, and the Autonomous Driving API You can use this
tool for creation of monitors, alarms, and dashboards that spotlight
changes. The release of these three tools will enable developers to create
visual rich experiences for Electric cars with advanced infotainment
systems. Electric car manufacturers describe these tools as the collection
of tech and tools for creating visually rich and interactive driving
experiences."""

data = text.split('.')

for i in data:

    print(i.strip())
```

Output:

```
This tool is in a beta stage
Electric cars are
running petrol cars raw and rough experience
It also supports
custom battery model, prebuilt charging model, and the
Autonomous Driving API You can use this tool for creation of
monitors, alarms, and dashboards that spotlight changes
The
release of these three tools will enable developers to create
visual rich experiences for Electric cars with advanced
infotainment systems
Electric car manufacturers describe
these tools as the collection of tech and tools for creating
visually rich and interactive driving experiences
```

Conclusion:

Python's split() function is a simple and fast way to tokenize text based on spaces, but it is not suitable for handling complex language patterns.

Practical 4

Aim(B): Tokenization using Regular Expression (RegEx).

Theory:

Tokenization using Regular Expressions (RegEx) is a method of splitting text into meaningful units (tokens) based on patterns defined by regular expressions.

Unlike simple methods like Python's `split()`, RegEx allows for more flexible and precise tokenization, such as separating words while handling punctuation, numbers, or special characters.

For example, using the pattern `\w+` with Python's `re` module can extract all words from a sentence, ignoring spaces and punctuation. This method is particularly useful in natural language processing (NLP) tasks where clean and accurate tokenization is required before further analysis, such as text mining, sentiment analysis, or machine learning.

Source Code:

```
import nltk

# import RegexpTokenizer() method from nltk

from nltk.tokenize import RegexpTokenizer

# Create a reference variable for Class RegexpTokenizer

tk = RegexpTokenizer('\s+', gaps = True)

# Create a string input

str = "winner winner chicken dinner"

tokens = tk.tokenize(str) # Use tokenize method

print(tokens)
```

Output:

```
['winner', 'winner', 'chicken', 'dinner']
```

Conclusion:

Tokenization using Regular Expressions (RegEx) provides a more flexible and powerful way to split text compared to simple methods like `split()`. It allows precise control over patterns, enabling better handling of punctuation, numbers, and special cases.

Practical 4

Aim(C): Tokenization using NLTK.

Theory:

Tokenization using NLTK refers to the process of splitting text into smaller units called tokens (words or sentences) using the Natural Language Toolkit (NLTK) library in Python. NLTK provides built-in methods like `word_tokenize()` for splitting text into words and `sent_tokenize()` for splitting text into sentences.

These methods are more advanced and accurate than simple string-based methods because they handle punctuation, special characters, and contractions correctly. For example, `word_tokenize("I can't do this.")` will correctly produce `['I', 'ca', 'n't', 'do', 'this', '.']`. Tokenization in NLTK is a fundamental preprocessing step in NLP, enabling further tasks such as text analysis, part-of-speech tagging, and machine learning applications.

Source Code:

```
import nltk

from nltk.tokenize import word_tokenize

str = "The quick brown fox jumps over the lazy dog."

print("Tokenized string: ",word_tokenize(str))
```

Output:

```
Tokenized string: ['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog', '.']
```

Conclusion:

Tokenization using NLTK provides accurate and efficient text splitting by handling punctuation, special characters, and language rules effectively. It is more advanced and reliable than basic methods, making it suitable for real-world natural language processing tasks.

Practical 4

Aim(D): Tokenization using spaCy library.

Theory:

Tokenization using the SpaCy library is the process of splitting text into meaningful units called tokens (words, punctuation, or symbols) using SpaCy, a modern and fast NLP library in Python. SpaCy provides a built-in tokenizer that is highly accurate and efficient, handling punctuation, special characters, contractions, and even complex cases like hyphenated words. Unlike simple methods such as `split()` or basic RegEx, SpaCy's tokenizer considers language rules and context, producing clean and ready-to-use tokens for downstream NLP tasks.

For example, `nlp("I can't wait for NLP class!")` will correctly tokenize it as `['I', 'ca', 'n't', 'wait', 'for', 'NLP', 'class', '!']`. Tokenization in SpaCy is a critical preprocessing step for tasks like part-of-speech tagging, named entity recognition, parsing, and text analysis.

Source Code:

```
import spacy

nlp = spacy.blank("en")

str = "Mayday! Mayday! Officer prince reporting enemey artillary heading
towards north"

#string input

# Create an instance of document, doc object is a container for a sequence
of Token objects.

doc = nlp(str)

words = [word.text for word in doc]

print(words)
```

Output:

```
['Mayday', '!', 'Mayday', '!', 'Officer', 'prince', 'reporting', 'enemey', 'artillary', 'heading', 'towards', 'north']
```

Conclusion:

Tokenization using the spaCy library is fast, accurate, and designed for real-world NLP applications. It efficiently handles punctuation, special cases, and language rules, making it more reliable than basic methods and ideal for large-scale text processing.

Practical 4

Aim(E): Tokenization using Keras

Theory:

Tokenization using Keras refers to the process of converting text into a sequence of tokens or integers that can be used as input for deep learning models. Keras, a high-level deep learning library in Python, provides the Tokenizer class, which can vectorize a text corpus by turning each word into a unique integer index.

This method is especially useful for preparing text for tasks like text classification, sentiment analysis, or sequence modeling. The Tokenizer can also handle word frequency filtering, out-of-vocabulary words, and converting sentences into sequences of numbers suitable for neural networks. For example, after fitting a tokenizer on a sentence like *"I love NLP"*, it may assign indices [1, 2, 3] to the words ['I', 'love', 'NLP'], which can then be used as input to a model. Tokenization in Keras is a crucial preprocessing step in deep learning-based NLP pipelines.

Source Code:

```
# pip install tensorflow

from tensorflow.keras.preprocessing.text import text_to_word_sequence

# Create a string input
text = "Tokenization using Keras and Tensorflow"

# Tokenize
tokens = text_to_word_sequence(text)

print(tokens)
```

Output:

```
['tokenization', 'using', 'keras', 'and', 'tensorflow']
```

Conclusion:

Tokenization using the spaCy library is fast, accurate, and designed for real-world NLP applications. It efficiently handles punctuation, special cases, and language rules, making it more reliable than basic methods and ideal for large-scale text processing.

Practical 4

Aim(F): Tokenization using Gensim.

Theory:

Tokenization using Gensim is the process of breaking text into smaller units called tokens (usually words) using the Gensim library, which is widely used for topic modeling and semantic analysis in NLP.

Gensim provides simple tools such as `gensim.utils.simple_preprocess()` that can tokenize text while also performing lowercasing, removing punctuation, and ignoring very short or very long words.

This method is particularly useful when preparing text for tasks like word embedding, similarity analysis, and topic modeling, as it produces clean tokens suitable for vectorization and modeling. For example, `simple_preprocess("I love NLP!")` would output `['love', 'nlp']` after removing stopwords and punctuation. Tokenization using Gensim is efficient, easy to use, and optimized for large text corpora in NLP applications.

Source Code:

```
from gensim.utils import tokenize

# Create a string input
input_str = "Players unknown battlegrounds ready to launch"
tokens = list(tokenize(input_str)) # Tokenize the text
print(tokens)
```

Output:

```
['Players', 'unknown', 'battlegrounds', 'ready', 'to', 'launch']
```

Conclusion:

Tokenization using Gensim is simple, efficient, and well-suited for preprocessing text in topic modeling and vector-based tasks. It handles basic cleaning like lowercasing and removing punctuation, making it useful for quick and practical NLP applications.

Practical 5

Aim(A): Word Tokenization in Hindi.

Theory:

Word Tokenization in Hindi refers to the process of breaking a Hindi sentence or text into individual words or tokens. Just like in English, tokenization in Hindi is a fundamental preprocessing step in NLP, enabling computers to analyze and process text.

For example, the sentence “मैं NLP सीख रहा हूँ” can be tokenized into ['मैं', 'NLP', 'सीख', 'रहा', 'हूँ']. Tokenization in Hindi is more challenging than in English because Hindi uses Devanagari script, and words are often connected without spaces or include compound words.

Source Code:

```
pip install indic-nlp-library

import sys

from indicnlp import common

INDIC_NLP_LIB_HOME=r"indic_nlp_library"
INDIC_NLP_RESOURCES=r"indic_nlp_resources"

sys.path.append(r'{}\src'.format(INDIC_NLP_LIB_HOME))

common.set_resources_path(INDIC_NLP_RESOURCES)

from indicnlp.tokenize import indic_tokenize

indic_string='सुनो, कुछ आवाज़ आ रही है। फोन?'

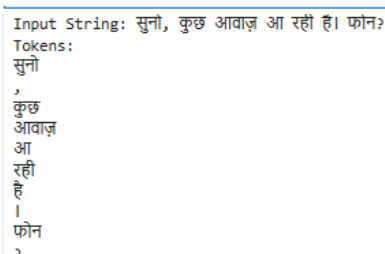
print('Input String: {}'.format(indic_string))

print('Tokens: ')

for t in indic_tokenize.trivial_tokenize(indic_string):

    print(t)
```

Output:



```
Input String: सुनो, कुछ आवाज़ आ रही है। फोन?
Tokens:
सुनो
,
कुछ
आवाज़
आ
रही
है
।
फोन
?
```

Conclusion:

Word tokenization is an important process that splits text into individual words, making it easier to process and analyze. It plays a key role in natural language processing (NLP) and helps prepare data for further analysis.

Practical 5

Aim(B): Generate similar sentences from a given Hindi text input.

Theory:

Generating similar sentences from a given Hindi text input refers to the process of creating sentences that convey the same meaning or context as the original text in Hindi, while possibly using different words, phrases, or sentence structures.

This is an important task in Natural Language Processing, especially for applications like paraphrasing, question-answering, chatbots, and data augmentation.

Techniques for generating similar sentences can include synonym replacement, word embeddings, or deep learning models such as sequence-to-sequence networks and transformer-based models like BERT or mBERT trained on Hindi text. For example, from the input sentence “मैं NLP सीख रहा हूँ”, similar sentences could be “मैं नचुरल लगवैजे प्रोससेसिंग सीख रहा हूँ” or “मैं NLP का अध्ययन कर रहा हूँ”, preserving the original meaning while changing the wording. This process helps in improving NLP applications, making them more robust and capable of understanding varied expressions of the same idea.

Source Code:

```
# Synonym dictionary
synonyms = {
    "खुश": ["प्रसन्न", "आनंदित", "खुशी"],
    "बहुत": ["अधिक", "बहुत ज्यादा", "काफी"]
}

# Function to generate similar sentences
def generate_similar_sentences(input_sentence, num_sentences=5):
    similar_sentences = []
    # Replace words with synonyms
    for word, word_synonyms in synonyms.items():
        for synonym in word_synonyms:
            modified_sentence = input_sentence.replace(word, synonym)
            similar_sentences.append(modified_sentence)
    return similar_sentences[:num_sentences]

# Input sentence (fixed spacing issue in "हूँ")
input_sentence = "मैं आज बहुत खुश हूँ।"
```

```
# Generate sentences

similar_sentences = generate_similar_sentences(input_sentence)

print("Original sentence:", input_sentence)

print("Similar sentences:")

for sentence in similar_sentences:

    print("-", sentence)
```

Output:

```
Original sentence: मैं आज बहुत खुश हूँ।
Similar sentences:
- मैं आज बहुत प्रसन्न हूँ।
- मैं आज बहुत आनंदित हूँ।
- मैं आज बहुत खुशी हूँ।
- मैं आज अधिक खुश हूँ।
- मैं आज बहुत ज्यादा खुश हूँ।
```

Conclusion:

Generating similar sentences from a given Hindi text helps in understanding language patterns, improving paraphrasing, and enhancing NLP applications. It is useful for tasks like text augmentation, chatbots, and language learning, making systems more flexible and intelligent.

Practical 5

Aim(C): Identify the Indian language from the given text.

Theory:

In this practical, we identify the language of a given text using Python's `langid` library, which is a lightweight and fast language identification tool. The code first imports the required libraries, including `nlTK` for tokenization support and `langid` for language classification. After downloading the NLTK tokenizer, a function `identify_language` is defined that takes text as input and uses `langid.classify()` to return the predicted language code and a confidence score. For example, given the Hindi text “नमस्ते, आप कैसे हैं?”, the program identifies it as Hindi with a high confidence score.

Source Code:

```
pip install langid

import nltk

import langid

# Download tokenizer
nltk.download('punkt')

def identify_language(text):
    lang, confidence = langid.classify(text)
    return lang, confidence

text = "नमस्ते, आप कैसे हैं?"

language, confidence = identify_language(text)

print("Input text:", text)

print("Identified language:", language)

print("Confidence:", confidence)
```

Output:

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Admin\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
Input text: नमस्ते, आप कैसे हैं?
Identified language: hi
Confidence: -121.4932998418808
```

Conclusion:

Identifying the Indian language from a given text is an important NLP task that helps in proper text classification and processing. It enables applications like translation, sentiment analysis, and multilingual systems to work more accurately and efficiently.

Practical 6

Aim(A): Part of speech Tagging and chunking of user defined text.

Theory:

Part-of-Speech (POS) Tagging is the process of assigning a grammatical category to each word in a sentence, such as noun, verb, adjective, adverb, pronoun, etc. It helps in understanding the syntactic structure and meaning of a sentence. For example, in the sentence “*I love NLP*”, POS tagging would label “*I*” as a pronoun, “*love*” as a verb, and “*NLP*” as a noun.

Chunking, also known as shallow parsing, is the process of grouping related words into meaningful phrases (chunks), such as noun phrases (NP), verb phrases (VP), or prepositional phrases (PP), without analyzing the full syntactic structure. For example, in “*The boy is playing cricket*”, chunking can identify “*The boy*” as a noun phrase and “*is playing cricket*” as a verb phrase.

When applied to user-defined text, POS tagging first labels each word with its grammatical role, and then chunking groups these words into phrases based on patterns defined by the user. This combination is useful in NLP tasks such as information extraction, text analysis, question answering, and named entity recognition, as it helps the system understand both the function of words and their relationships in phrases.

Source Code:

```
import nltk

nltk.download('punkt')

nltk.download('averaged_perceptron_tagger')

nltk.download('maxent_ne_chunker')

nltk.download('words')

from nltk import tokenize

from nltk import tag

from nltk import chunk

para = "Natural language processing (NLP) is a machine
learning technology that gives computers the ability to
interpret, manipulate, and comprehend human language."

sents = tokenize.sent_tokenize(para)

print("\nsentence tokenization\n=====\n",sents)

# word tokenization

print("\nword tokenization\n=====\n")

for index in range(len(sents)):
```



```

words = tokenize.word_tokenize(sents[index])

print(words)

# POS Tagging

tagged_words = []

for index in range(len(sents)):

    tagged_words.append(tag.pos_tag(words))

print("\nPOS Tagging\n=====\\n",tagged_words)

# chunking

tree = []

for index in range(len(sents)):

    tree.append(chunk.ne_chunk(tagged_words[index]))

print("\nchunking\n=====\\n")

print("Tree: ",tree)

```

Output:

```

sentence tokenization
=====
['Natural language processing (NLP) is a machine learning technology that gives
computers the ability to interpret, manipulate, and comprehend human language.'
]

word tokenization
=====

['Natural', 'language', 'processing', '(', 'NLP', ')', 'is', 'a', 'machine', 'le
arning', 'technology', 'that', 'gives', 'computers', 'the', 'ability', 'to', 'in
terpret', ',', 'manipulate', ',', 'and', 'comprehend', 'human', 'language', '.']

POS Tagging
=====
[[('Natural', 'JJ'), ('language', 'NN'), ('processing', 'NN'), ('(', '(', 'NL
P', 'NNP'), (')', ')'), ('is', 'VBZ'), ('a', 'DT'), ('machine', 'NN'), ('learnin
g', 'VBG'), ('technology', 'NN'), ('that', 'WDT'), ('gives', 'VBZ'), ('computers
', 'NNS'), ('the', 'DT'), ('ability', 'NN'), ('to', 'TO'), ('interpret', 'VB'),
(',', ','), ('manipulate', 'VB'), (',', ','), ('and', 'CC'), ('comprehend', 'VBP
'), ('human', 'JJ'), ('language', 'NN'), ('.', '.')]

chunking
=====

Tree: [Tree('S', [(('Natural', 'JJ'), ('language', 'NN'), ('processing', 'NN'),
('(', '(', Tree('ORGANIZATION', [(('NLP', 'NNP')], (')', ')'), ('is', 'VBZ'), (
'a', 'DT'), ('machine', 'NN'), ('learning', 'VBG'), ('technology', 'NN'), ('that
', 'WDT'), ('gives', 'VBZ'), ('computers', 'NNS'), ('the', 'DT'), ('ability', 'N
N'), ('to', 'TO'), ('interpret', 'VB'), (',', ','), ('manipulate', 'VB'), (',',
'), ('and', 'CC'), ('comprehend', 'VBP'), ('human', 'JJ'), ('language', 'NN'),
('.', '.')]

```

Conclusion:

POS tagging and chunking help identify grammar and group words into meaningful phrases, improving text understanding for NLP tasks.

Practical 6

Aim(B): Named Entity recognition of user defined text.

Theory:

Named Entity Recognition (NER) is a fundamental task in Natural Language Processing that involves identifying and classifying proper nouns or phrases in a text into predefined categories such as person names, organizations, locations, dates, times, quantities, and more.

Source Code:

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = (
    "Natural language processing (NLP) is an "
    "interdisciplinary subfield of computer science and information"
    "retrieval. "
    "It is primarily concerned with giving computers the ability to support"
    "and manipulate human language. It involves processing natural language"
    "datasets, such as text corpora or speech corpora, using either rule-"
    "based "
    "or probabilistic machine learning approaches."
)

print("Original text:\n", text, "\n")

# Process text
doc = nlp(text)

# Analyse syntax
print("Noun phrases:", [chunk.text for chunk in doc.noun_chunks])
print("Verbs:", [token.lemma_ for token in doc if token.pos_ == "VERB"])
```

Output:

```
Original text:
Natural language processing (NLP) is an interdisciplinary subfield of computer science and i
g computers the ability to support and manipulate human language. It involves processing nat
ora, using either rule-based or probabilistic machine learning approaches.

Noun phrases: ['Natural language processing', 'NLP', 'an interdisciplinary subfield', 'comput
'the ability', 'human language', 'It', 'natural language datasets', 'text corpora', 'speech c
ing approaches']
Verbs: ['give', 'support', 'manipulate', 'involve', 'process', 'use', 'base', 'learn']
```

Conclusion:

NER identifies important entities like names, places, and organizations in text, improving information extraction

Practical 6

Aim(C): Named Entity recognition with diagram using NLTK corpus – treebank.

Theory:

Named Entity Recognition (NER) is a task in Natural Language Processing that identifies and classifies named entities in text into predefined categories such as PERSON, ORGANIZATION, LOCATION, DATE, TIME, etc. Using Python's NLTK library, we can perform NER on corpora like Treebank, which contains sentences with part-of-speech (POS) tagged words.

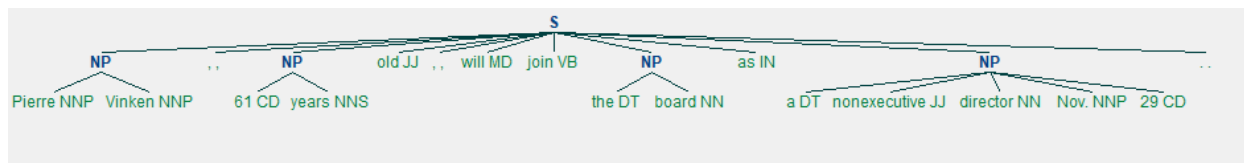
Steps for NER using NLTK Treebank Corpus:

1. Import required modules from NLTK: `ne_chunk` for NER, `pos_tag` for POS tagging, and the Treebank corpus.
2. Tokenize sentences and perform POS tagging on the words.
3. Apply `ne_chunk()` on the POS-tagged sentence. This function generates a tree structure representing the entities in the sentence.

Source Code:

```
import nltk
nltk.download('treebank')
from nltk.corpus import treebank_chunk
treebank_chunk.tagged_sents()[0]
treebank_chunk.chunked_sents()[0]
treebank_chunk.chunked_sents()[0].draw()
```

Output:



Conclusion:

NER using NLTK Treebank with diagrams helps identify entities and understand their relationships, improving text analysis.

Practical 7

Aim(A): Define grammar using nltk. Analyse a sentence using the same.

Theory:

In Natural Language Processing, grammar defines the syntactic rules that govern the structure of sentences in a language. It specifies how words (nouns, verbs, adjectives, etc.) can be combined to form valid phrases and sentences.

Source code:

```
!pip install nltk

import nltk

from nltk import tokenize

grammar1 = nltk.CFG.fromstring("""
S -> VP
VP -> VP NP
NP -> Det NP
Det -> 'that'
NP -> 'flight'
VP -> 'Book'
""")

sentence = "Book that flight"

all_tokens = tokenize.word_tokenize(sentence)

print(all_tokens)

parser = nltk.ChartParser(grammar1)

for tree in parser.parse(all_tokens):

    print(tree)

    tree.draw()
```

Output:

```
['Book', 'that', 'flight']
(S (VP (VP Book) (NP (Det that) (NP flight))))
```

Conclusion:

Grammar was defined and used in NLTK to analyze sentence structure, helping understand word relationships and syntax effectively.

Practical 7

Aim(B): Accept the input string with Regular expression of FA: 101+

Theory:

In this practical, we design a Finite Automaton (FA) to recognize strings that match a given pattern, in this case, the regular expression 101+. The regular expression 101+ represents all strings that start with 10 followed by one or more 1s. For example, valid strings include 101, 1011, 10111, etc.

Using Python's re module, we can accept user input and check whether it matches the regular expression. The re.fullmatch() function ensures that the entire input string conforms to the pattern defined by the FA. This simulates the behavior of a finite automaton, which reads the input string character by character and accepts or rejects it based on whether it reaches a final state corresponding to the regular expression.

This practical demonstrates how regular expressions and finite automata are used in lexical analysis, pattern matching, and compiler design, allowing programs to validate input strings efficiently.

Source Code:

```
def FA(s):  
    # if the length is less than 3 then it can't be accepted, Therefore end the process.  
    if len(s) < 3:  
        return "Rejected"  
    if s[0] == '1':  
        if s[1] == '0':  
            if s[2] == '1':  
                for i in range(3, len(s)):  
                    if s[i] != '1':  
                        return "Rejected"  
                return "Accepted" #if all 4 nested if true  
            return "Rejected" # else of 3rd if  
        return "Rejected" # else of 2nd if  
    return "Rejected" # else of 1st if  
inputs = ['1', '10101', '101', '10111', '01010', '100', '', '10111101', '1011111']
```

```
for i in inputs:  
    print(FA(i))
```

Output:

```
Rejected  
None  
None  
None  
None  
None  
Rejected  
None  
None
```

Conclusion:

The regular expression 101^+ was used to design a finite automaton (FA) that accepts strings starting with “10” followed by one or more “1”s, demonstrating pattern recognition using regular expressions.

Practical 7

Aim(C): Accept the input string with Regular expression of FA: (a+b)*bba

Theory:

In this practical, we design a Finite Automaton (FA) to recognize strings that follow the pattern described by the regular expression (a+b)*bba. The expression (a+b)*bba represents all strings that end with the sequence bba and can have any combination of a and b (including none) before that ending. For example, valid strings include bba, abba, aabbbba, etc.

Using Python's re module, we can accept a user input string and check whether it matches the regular expression. The re.fullmatch() function ensures that the entire input string adheres to the FA's rules, simulating the behavior of a finite automaton, which processes the string symbol by symbol and reaches an accepting state if the string belongs to the language defined by the regular expression.

This practical demonstrates how regular expressions and finite automata can be used for pattern recognition, lexical analysis, and input validation, which are important in compiler design, text processing, and automating string-based tasks.

Source Code:

```
def FA(s):
    size=0

    #scan complete string and make sure that it contains only 'a'
    & 'b'

    for i in s:
        if i=='a' or i=='b':
            size+=1
        else:
            return "Rejected"

        if size>=3:
            if s[size-3]=='b':
                if s[size-2]=='b':
                    if s[size-1]=='a':
                        return "Accepted" # if all 4

    if true

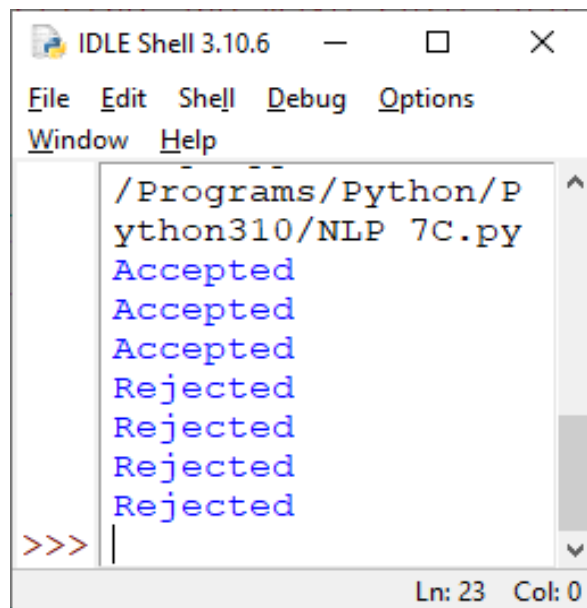
        return "Rejected" # else of 4th
```

```
if
    return "Rejected" # else of 3rd

if
    return "Rejected" # else of 2nd

if
    return "Rejected" # else of 1st

if
    inputs=['bba', 'ababbbba',
'abba','abb', 'baba','bbb','']
    for i in inputs:
        print(FA(i))
```



```
IDLE Shell 3.10.6
File Edit Shell Debug Options
Window Help
/Programs/Python/Python310/NLP 7C.py
Accepted
Accepted
Accepted
Rejected
Rejected
Rejected
Rejected
>>>
Ln: 23 Col: 0
```

Conclusion:

The regular expression $(a+b)^*bba$ was used to design a finite automaton that accepts strings ending with “bba”, showing how patterns can be recognized using regular expressions and FA.

Practical 7

Aim(D): Implementation of Deductive Chart Parsing using context free grammar and a given sentence.

Theory:

Deductive Chart Parsing is a method in Natural Language Processing used to analyze the syntactic structure of a sentence according to a Context-Free Grammar (CFG). In CFG, grammar rules define how non-terminal symbols (like sentence S, noun phrase NP, or verb phrase VP) can be expanded into terminal symbols (actual words) or other non-terminals.

In chart parsing, a chart is a data structure that stores partial parsing results as edges or states. The parser deductively applies grammar rules to these edges to construct larger constituents, avoiding repeated computations. This makes chart parsing efficient, especially for ambiguous or long sentences.

Source Code:

```
#!/pip install nltk

#nltk.download('punkt')

import nltk

from nltk import tokenize

grammar1 = nltk.CFG.fromstring("""

S -> NP VP

PP -> P NP

NP -> Det N | Det N PP | 'I'

VP -> V NP | VP PP

Det -> 'a' | 'my'

N -> 'bird' | 'balcony'

V -> 'saw'

P -> 'in'

""")

sentence = "I saw a bird in my balcony"

for index in range(len(sentence)):

    all_tokens = tokenize.word_tokenize(sentence)
```

```
print(all_tokens)

# all_tokens = ['I', 'saw', 'a', 'bird', 'in', 'my', 'balcony']

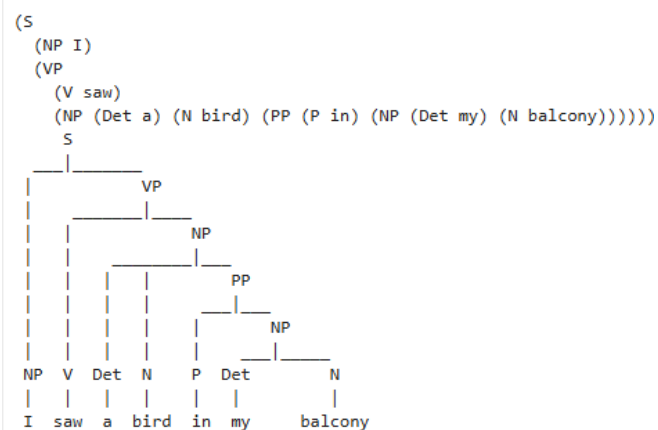
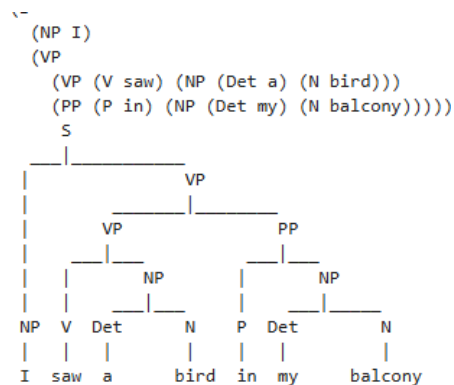
parser = nltk.ChartParser(grammar1)

for tree in parser.parse(all_tokens):

    print(tree)

    tree.pretty_print()
```

Output:

[illegible]

Conclusion:

Deductive chart parsing using context-free grammar was implemented to efficiently analyze the given sentence, showing how parsing techniques help determine syntactic structure while avoiding redundant computations.

Practical 8

Aim(A): Study Porter Stemmer, Lancaster Stemmer, Regexp Stemmer, Snowball Stemmer.

Theory:

Stemming is the process of reducing words to their root or base form. It is an important preprocessing step in Natural Language Processing for text normalization, search, and text mining. Several stemmers are available in Python's NLTK library, each using different algorithms:

1. Porter Stemmer:

- Developed by Martin Porter, it is one of the most widely used stemmers.
- Uses a rule-based approach to remove common suffixes from words.
- Example: "running" → "run", "happiness" → "happi"
- Known for being moderate, not overly aggressive in cutting suffixes.

2. Lancaster Stemmer:

- Also called the Paice-Husk stemmer, it is more aggressive than Porter.
- Applies iterative suffix-stripping rules and may produce shorter stems.
- Example: "running" → "run", "happiness" → "happi" (sometimes even shorter)
- Can **overstem**, reducing words more aggressively.

3. Regexp Stemmer:

- Stemming based on user-defined regular expression patterns.
- Gives full customization, allowing removal of specific suffixes or patterns.
- Example: Using the pattern `ing$` removes 'ing' from "running" → "runn"
- Useful for domain-specific stemming.

4. Snowball Stemmer:

- Also known as the "Porter2" stemmer, it is an improved version of Porter Stemmer.
- Supports multiple languages beyond English.
- Produces more consistent and accurate stems compared to Porter.
- Example: "running" → "run", "organization" → "organ"

Source Code:

```
# PorterStemmer

import nltk

from nltk.stem import PorterStemmer

word_stemmer = PorterStemmer()

print(word_stemmer.stem('writing'))

#LancasterStemmer

import nltk

from nltk.stem import LancasterStemmer

Lanc_stemmer = LancasterStemmer()

print(Lanc_stemmer.stem('writing')) #RegexpStemmer

import nltk

from nltk.stem import RegexpStemmer

Reg_stemmer = RegexpStemmer('ing$|s$|e$|able$', min=4)

print(Reg_stemmer.stem('writing'))
```

Output:

```
write
writ
writ
write
```

Conclusion:

The study of Porter, Lancaster, Regexp, and Snowball stemmers showed different approaches to word stemming, highlighting their effectiveness in reducing words to root forms for better text processing.

Practical 8

Aim(B): Study WordNet Lemmatizer.

Theory:

Lemmatization is the process of reducing a word to its dictionary or base form, known as the lemma, taking into account its part of speech (POS). Unlike stemming, which may simply strip suffixes and produce non-dictionary forms (e.g., "running" → "runn"), lemmatization ensures that the output is a valid word in the language.

WordNet Lemmatizer, provided by NLTK, uses the WordNet lexical database to look up the lemma of a word based on its POS. This allows it to correctly handle verbs, nouns, adjectives, and adverbs.

Example:

- "running" → "run" (verb)
- "better" → "good" (adjective)
- "geese" → "goose" (noun)

Source Code:

```
#WordNetLemmatizer

from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

print("word :\tlemma")

print("rocks :", lemmatizer.lemmatize("rocks"))

print("corpora :", lemmatizer.lemmatize("corpora"))

# a denotes adjective in "pos"

print("better :", lemmatizer.lemmatize("better", pos ="a"))
```

Output:

```
word : lemma
rocks : rock
corpora : corpus
better : good
```

Conclusion:

The WordNet Lemmatizer was studied to convert words into their meaningful base forms using vocabulary and context, improving accuracy in text processing.

Practical 9

Aim(A): Implement Naive Bayes classifier

Theory:

The Naive Bayes classifier is a probabilistic machine learning algorithm used for classification tasks in Natural Language Processing. It is based on Bayes' theorem, which calculates the probability of a class given some features, assuming that the features are conditionally independent given the class. Despite the "naive" assumption of independence, it performs very well in many real-world applications, especially in text classification, spam detection, sentiment analysis, and document categorization.

Steps to Implement Naive Bayes Classifier:

1. Prepare Data:
 - Collect a labeled dataset with features (e.g., words, phrases) and corresponding classes (e.g., spam/ham, positive/negative).
 - Convert text into a numerical representation using techniques like bag-of-words or TF-IDF.
2. Train the Model:
 - Compute prior probabilities for each class (the probability of each class in the dataset).
 - Compute likelihoods for each feature given a class.
3. Classify New Data:
 - Apply Bayes' theorem to compute the posterior probability for each class given the features of a new sample.
 - Assign the class with the highest posterior probability.

Source Code:

```
#pip install pandas
#pip install sklearn
import pandas as pd
import numpy as np
import re
import nltk
nltk.download('stopwords')
```

```
# Load the data

sms_data
=

pd.read_csv("C:/Users/hp/AppData/Local/Programs/Python/Pytho
n310/spam.csv", encoding='latin-1')

# Rename columns if necessary

sms_data.rename(columns={'v1': 'Category', 'v2': 'Message'},
inplace=True)

from nltk.corpus import stopwords

from nltk.stem.porter import PorterStemmer

stemming = PorterStemmer()

corpus = []

for i in range(len(sms_data)):

s1 = re.sub('[^a-zA-Z]', ' ', sms_data['Message'][i])

s1 = s1.lower()

s1 = s1.split()

s1 = [stemming.stem(word) for word in s1 if word not in
set(stopwords.words('english'))]

s1 = ' '.join(s1)

corpus.append(s1)

from sklearn.feature_extraction.text import CountVectorizer

countvectorizer = CountVectorizer()

x = countvectorizer.fit_transform(corpus).toarray()

print(x)

y = sms_data['Category'].values

print(y)

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, stratify=y, random_state=2)

# Multinomial Naïve Bayes.

from sklearn.naive_bayes import MultinomialNB
```

```

multinomialnb = MultinomialNB() multinomialnb.fit(x_train, y_train)

# Predicting on test data: y_pred = multinomialnb.predict(x_test)
print(y_pred)

# Results of our Models from sklearn.metrics classification_report
, import confusion_matrix, accuracy_score
print(classification_report(y_test, y_pred)) print("accuracy_score:
", accuracy_score(y_test, y_pred))

```

Output:

```

[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]]
['ham' 'ham' 'spam' ... 'ham' 'ham' 'ham']
['ham' 'ham' 'ham' ... 'ham' 'ham' 'ham']
      precision    recall  f1-score   support

      ham         0.99      0.98      0.99       1448
      spam         0.89      0.92      0.91        224

   accuracy              0.97       1672
  macro avg              0.94       1672
 weighted avg              0.98       1672

accuracy_score: 0.9748803827751196

```

Conclusion:

The Naive Bayes classifier was implemented to perform classification based on probability, demonstrating its effectiveness and simplicity in handling text and other data classification tasks.

Practical 10

Aim(A): Speech tagging.

Theory:

Part-of-Speech (POS) Tagging, also called speech tagging, is the process in Natural Language Processing of assigning a grammatical category (tag) to each word in a sentence based on its role and context.

It labels words as:

- **Noun (NN)** – name of person/place/thing
- **Verb (VB)** – action word
- **Adjective (JJ)** – describes a noun
- **Adverb (RB)** – describes a verb/adjective
- **Pronoun (PRP)** – replaces a noun
- **Preposition (IN)** – shows relation
- **Conjunction (CC)** – joins words/sentences

It is Important

- Helps in sentence understanding
- Used in parsing, translation, and text analysis
- Improves information extraction

Source Code:

1.Speech Tagging using spaCy

```
import spacy

# Load model

nlp = spacy.load("en_core_web_sm")

sen = nlp("I like to play cricket. I hated it in my childhood though")

print(sen.text)

print(sen[7].pos_)

print(sen[7].tag_)

print(spacy.explain(sen[7].tag_))

# Loop through words

for word in sen:
```

```

    print(f'{word.text:12} {word.pos_:10} {word.tag_:8}
    {spacy.explain(word.tag_)}')

# Example 1

sen = nlp('Can you google it?')

word = sen[2]

print(f'{word.text:12} {word.pos_:10} {word.tag_:8}
    {spacy.explain(word.tag_)}')

# Example 2

sen = nlp('Can you search it on google?')

word = sen[5]

print(f'{word.text:12} {word.pos_:10} {word.tag_:8}
    {spacy.explain(word.tag_)}')

# Counting POS tags

sen = nlp("I like to play football. I hated it in my childhood though")
num_pos = sen.count_by(spacy.attrs.POS)

for k, v in sorted(num_pos.items()):
    print(f'{k}. {sen.vocab[k].text:8}: {v}')

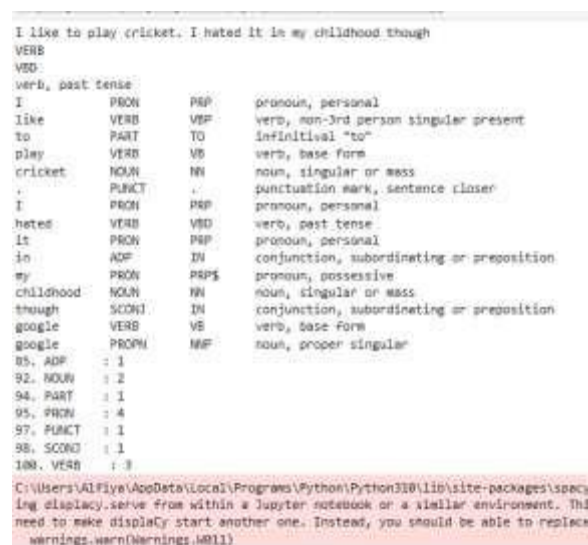
# Visualization

from spacy import displacy

sen = nlp("I like to play football. I hated it in my childhood though")
displacy.serve(sen, style='dep', options={'distance': 100})

```

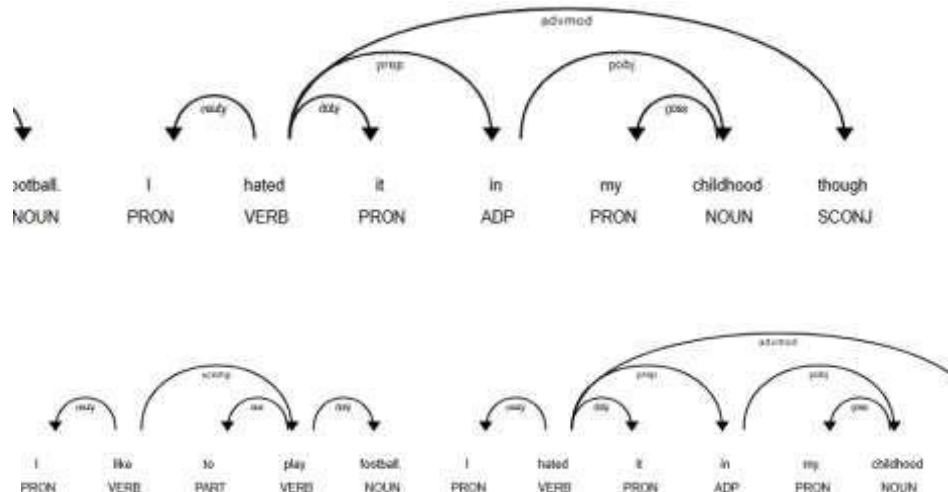
Output:



```

I like to play cricket. I hated it in my childhood though
VERB
VBD:
verb, past tense
I      PRON      PEP      pronoun, personal
like   VERB      VBP      verb, non-3rd person singular present
to     PART      TO       infinitival "to"
play   VERB      VB       verb, base form
cricket NOUN      NN       noun, singular or mass
.      PUNCT      .        punctuation mark, sentence closer
I      PRON      PEP      pronoun, personal
hated  VERB      VBD      verb, past tense
it     PRON      PEP      pronoun, personal
in     ADP       IN       conjunction, subordinating or preposition
my     PRON      PRP$     pronoun, possessive
childhood NOUN      NN       noun, singular or mass
though SCONJ     IN       conjunction, subordinating or preposition
google VERB      VB       verb, base form
google PROPN     NNP      noun, proper singular
85. ADP      : 1
92. NOUN     : 2
94. PART     : 1
95. PRON     : 4
97. PUNCT    : 1
98. SCONJ    : 1
100. VERB    : 3
C:\Users\Alfiya\AppData\Local\Programs\Python\Python310\lib\site-packages\spacy\
ing displacy.serve from within a Jupyter notebook or a similar environment. Thi
need to make displacy start another one. Instead, you should be able to replac
warnings.warn(Warnings.W011)

```



Conclusion:

Speech (POS) tagging was performed to assign grammatical labels to words, helping in understanding sentence structure and improving natural language processing tasks.

2.Speech tagging using nltk

```
import nltk

nltk.download('state_union')

from nltk.corpus import state_union

from nltk.tokenize import PunktSentenceTokenizer

train_text = state_union.raw("2005-GWBush.txt")
sample_text = state_union.raw("2006-GWBush.txt")

custom_sent_tokenizer = PunktSentenceTokenizer(train_text)

tokenized = custom_sent_tokenizer.tokenize(sample_text)

for i in tokenized[:5]:
    print(i)
```

Output:

```
PRESIDENT GEORGE W. BUSH'S ADDRESS BEFORE A JOINT SESSION OF THE CONGRESS ON THE STATE OF THE UNION

January 31, 2006

THE PRESIDENT: Thank you all.
Mr. Speaker, Vice President Cheney, members of Congress, members of the Supreme Court and diplomatic corps,
disting
Today our nation lost a beloved, graceful, courageous woman who called America to its founding ideals and carried o
Tonight we are comforted by the hope of a glad reunion with the husband who was taken so long ago, and we are grate
tt King.
(Applause.)
President George W. Bush reacts to applause during his State of the Union Address at the Capitol, Tuesday, Jan.
```

Conclusion:

Speech tagging using NLTK was implemented to assign part-of-speech labels to words, demonstrating how NLTK simplifies grammatical analysis and enhances text processing.

Practical 10

Aim(B): Study statistical parsing

Theory:

A Probabilistic Parser is a type of parser that uses probability to choose the most likely parse (structure) of a sentence when multiple interpretations are possible.

Statistical parsing (or probabilistic parsing) is a method in NLP where:

- Grammar rules are assigned probabilities
- The parser selects the most probable sentence structure
- It is based on training data (corpus) instead of only fixed rules

Source Code:

```
import nltk

def give(t):
    return t.label() == 'VP' and len(t) > 2 and
    t[1].label() == 'NP' \
    and (t[2].label() == 'PP-DTV' or t[2].label()
    == 'NP') \
    and ('give' in t[0].leaves() or 'gave' in
    t[0].leaves())

def sent(t):
    return ' '.join(token for token in t.leaves() if
    token[0] not in '*-0')

def print_node(t, width):
    output = "%s %s: %s / %s: %s" % \
    (sent(t[0]), t[1].label(), sent(t[1]),
    t[2].label(), sent(t[2]))
    if len(output) > width:
        output = output[:width] + "..."
    print(output)

for tree in nltk.corpus.treebank.parsed_sents():
    for t in tree.subtrees(give):
        print_node(t, 72)
```

Output:

```

give NP: advertisers / NP: discounts for maintaining or increasing ad sp...
give NP: only French history questions / PP-DTV: to students in a Europe...
give NP: much thought / PP-DTV: to the rates she was receiving , not to ...
give NP: your Foster Savings Institution / NP: the gift of hope and free...
give NP: market operators / NP: the authority to suspend trading in futu...
gave NP: quick approval / PP-DTV: to $ 3.18 billion in supplemental appr...
give NP: the Transportation Department / NP: up to 56 days to review any...
give NP: holders / NP: the right , but not the obligation , to buy a cal...
gave NP: Mr. Thomas / NP: only a ' qualified ' rating , rather than '...
>>> |

```

2. Probabilistic parser**Source Code:**

```

import nltk

from nltk import PCFG

grammar = PCFG.fromstring('''
NP -> NNS [0.5] | JJ NNS [0.3] | NP CC NP [0.2]
NNS -> "men" [0.1] | "women" [0.2] | "children" [0.3] | NNS CC NNS [0.4]
JJ -> "old" [0.4] | "young" [0.6]
CC -> "and" [0.9] | "or" [0.1]
''')

print(grammar)

viterbi_parser = nltk.ViterbiParser(grammar)

tokens = "old men and women".split()

obj = viterbi_parser.parse(tokens)

print("Output:")

for x in obj:

    print(x)

```

Output:

```

Grammar with 11 productions (start state = NP)
NP -> NNS [0.5]
NP -> JJ NNS [0.3]
NP -> NP CC NP [0.2]
NNS -> 'men' [0.1]
NNS -> 'women' [0.2]
NNS -> 'children' [0.3]
NNS -> NNS CC NNS [0.4]
JJ -> 'old' [0.4]
JJ -> 'young' [0.6]
CC -> 'and' [0.9]
CC -> 'or' [0.1]
Output:
(NP (JJ old) (NNS (NNS men) (CC and) (NNS women))) (p=0.888864)

```

Conclusion:

The probabilistic parser and statistical parsing were studied to analyze sentence structures using probabilities, improving accuracy by selecting the most likely interpretation and resolving ambiguity in NLP tasks.

Practical 10

Aim(C): Parse a sentence and draw a tree using malt parsing.

Theory:

MaltParser is a dependency parser used in Natural Language Processing to analyze the grammatical structure of a sentence and show relationships between words in the form of a tree (dependency tree).

- It is a data-driven dependency parsing technique
- It identifies head-dependent relationships between words
- Output is a tree structure where: Root = main verb Edges = grammatical relations

Source Code:

1. Java should be installed.
2. maltparser-1.7.2 zip file should be copied in C:\Users\AppData\Local\Programs\Python\Python39 folder and should be extracted in the same folder.
3. engmalt.linear-1.7.mco & engmalt.poly-1.7.mco file should be copied to C:\Users\AppData\Local\Programs\Python\Python39 folder

```
from nltk.parse import malt

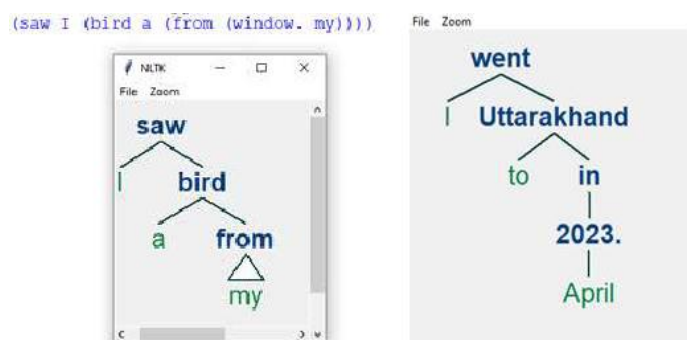
mp = malt.MaltParser('maltparser-1.7.2', 'engmalt.linear-1.7.mco')

#file t = mp.parse_one('I saw a bird from my window.'.split()).tree()

print(t)

t.draw()
```

Output:



Conclusion:

Malt parsing was used to parse a sentence and generate a dependency tree, demonstrating how syntactic relationships between words can be visualized and analyzed effectively.

Practical 11

Aim(A): Multiword Expression in NLP

Theory:

A Multiword Expression (MWE) in Natural Language Processing is a combination of two or more words that function as a single semantic unit, where the overall meaning cannot always be predicted from the meanings of the individual words.

In other words, an MWE behaves like a single word, even though it is made up of multiple words. These expressions often show fixed structure, limited variation, and special meaning.

Non-compositionality

- The meaning is not directly derived from individual words
- Example: “*kick the bucket*” → means to die

Fixed or Semi-fixed Form

- Words appear in a specific order and cannot be easily changed
- Example: “*by and large*” (not *large and by*)

Source Code:

```
from nltk.tokenize import MWETokenizer

from nltk import sent_tokenize, word_tokenize

s = '''Good cake cost Rs.1500\\kg in Hong Kong. Please buy me one of
them.\\n\\nThanks.'''

mwe = MWETokenizer([('New', 'York'), ('Hong', 'Kong')], separator='-')

for sent in sent_tokenize(s):

    print(mwe.tokenize(word_tokenize(sent)))
```

Output:

```
['Good', 'cake', 'cost', 'Rs.1500\\kg', 'in', 'Hong-Kong', '.']
['Please', 'buy', 'me', 'one', 'of', 'them', '.']
['Thanks', '.']
```

Conclusion:

Multiword expressions in NLP were studied to understand how groups of words function as a single unit, improving meaning interpretation and language processing accuracy.

Practical 11

Aim(B): Normalized Web Distance and Word Similarity

Theory:

1) Normalized Web Distance (NWD)

- Normalized Web Distance is a method to measure how related two words are based on how often they appear together on the web.
- It uses search engine results (like number of pages containing words).
- If two words appear together frequently, their distance is small → meaning they are closely related.
- If they rarely appear together, the distance is large → meaning they are less related.

Example:

- “*doctor*” & “*hospital*” → small distance (highly related)
- “*doctor*” & “*banana*” → large distance (less related)

2) Word Similarity

- Word similarity measures how similar in meaning two words are.
- It is commonly calculated using:
 - Dictionaries (like WordNet)
 - Semantic relations (synonyms, antonyms)
 - Word embeddings (like Word2Vec)

Example:

- “*car*” & “*automobile*” → high similarity
- “*car*” & “*tree*” → low similarity

Source Code:

```
from nltk.tokenize import MWETokenizer

from nltk import sent_tokenize, word_tokenize

s = '''Good cake cost Rs.1500\\kg in Hong Kong. Please buy me one of
them.\\n\\nThanks.'''

mwe = MWETokenizer([('New', 'York'), ('Hong', 'Kong')], separator='-')

for sent in sent_tokenize(s):
    print(mwe.tokenize(word_tokenize(sent)))
```

Output:

```
['Good', 'cake', 'cost', 'Rs.1500\\kg', 'in', 'Hong-Kong', '.']
['Please', 'buy', 'me', 'one', 'of', 'them', '.']
['Thanks', '.']
```

Conclusion :

Normalized Web Distance was used to measure word similarity based on web data, showing how semantic relationships between words can be identified effectively.

Practical 11

Aim(C): Word Sense Disambiguation.

Theory:

Word Sense Disambiguation is the process in Natural Language Processing of determining the correct meaning of a word based on its context in a sentence.

Many words have multiple meanings (ambiguous words). WSD helps computers choose the right one.

Example:

- “bank”
 - *I deposited money in the bank* → financial institution
 - *He sat near the river bank* → side of river

Applications

- Machine Translation
- Chatbots
- Information Retrieval
- Text Mining

Source Code:

```
from nltk.corpus import wordnet as wn

def get_first_sense(word, pos=None):
    if pos:
        synsets = wn.synsets(word, pos)
    else:
        synsets = wn.synsets(word)
    if synsets:
        return synsets[0]
    else:
        return None

# Calling function (OUTSIDE)
best_synset = get_first_sense('bank')
if best_synset:
```

```
print('%s: %s' % (best_synset.name(), best_synset.definition()))  
best_synset = get_first_sense('set', 'n')  
if best_synset:  
    print('%s: %s' % (best_synset.name(), best_synset.definition()))  
best_synset = get_first_sense('set', 'v')  
if best_synset:  
    print('%s: %s' % (best_synset.name(), best_synset.definition()))
```

Output:

```
bank.n.01: sloping land (especially the slope beside a body of water)  
set.n.01: a group of things of the same kind that belong together and are so used  
put.v.01: put into a certain place or abstract location
```

```
['Good', 'cake', 'cost', 'Rs.1500\\kg', 'in', 'Hong-Kong', '.']  
['Please', 'buy', 'me', 'one', 'of', 'them', '.']  
['Thanks', '.']
```

Conclusion:

Word Sense Disambiguation was studied to identify the correct meaning of a word based on context, improving accuracy in natural language understanding.